KERALA POLYTECHNIC · REVISION 2021

Introduction to Python

Complete student lesson for course code **1257**.

Revision 2021 Semester 1 Program Basic 4 modules

Course snapshot

Scheme: 3 (L:1, T:0, P:2)

Credits: 2

Contact: 45 periods per semester

Module hours listed: 42

Official Source Declaration

This lesson uses the supplied official syllabus PDF as the authoritative source for course information, revision, modules, topics, objectives, Course Outcomes, assessment information, official activities, practical requirements, and references.

Source handling: Official wording and numerical values are retained wherever supplied. Educational explanations, Malayalam support, examples, diagrams, and revision questions are added as study support and are not presented as additional official requirements.

Transparency note: Official assessment information is reproduced below from the supplied syllabus. Any limitation in the supplied PDF is not silently corrected.

How to Study This Lesson

Recommended sequence

1. Begin with the official source declaration and course dashboard.

2. Study each module in the official order and keep the coverage table as a checklist.
3. Open every topic, understand the example or procedure, and write your own short answer.
4. Use the revision section, glossary, questions, and practice paper after completing the modules.

Study principle: Keep English technical terminology, symbols, units, and official assessment wording unchanged in examination answers. Use the Malayalam support blocks only as a bridge to understanding.

Handbook method: Do not treat a topic as completed after reading its heading. For every topic card, complete the learning objectives, follow the conceptual framework, write the worked answer method, and answer the self-check questions. This creates a study record that is useful for class learning and examination preparation.

Course Dashboard

Course code

1257

Semester

1

Credits

2

Teaching scheme

3 (L:1, T:0, P:2)

Module-hour and outcome mapping

No.	Module	Official hours	Relevant CO / level
1	Python Syntax, Operator Precedence and Control Flow	10	CO1
2	Functions and Strings	12	CO2

No.	Module	Official hours	Relevant CO / level
3	Lists, Dictionaries, Tuples and Sets	10	CO3
4	Files and Exception Handling	10	CO4

Prerequisites

- Basic understanding of computer terminologies (High School).

How to study this lesson

1. Read the module introduction and learning goals.
2. Open every topic and reproduce its example, sequence, or procedure.
3. Use Malayalam support as a bridge; retain English technical terms for examinations.
4. Attempt the module questions without looking at the answers.

Objectives, Course Outcomes and CO–PO Information

Official course objectives

1. To provide a hands-on experience in programming concepts using Python Programming Language. Students will be able to develop programs using arrays, functions, and files.
2. To enhance the problem-solving skills of students.

Course Outcomes

1. Apply the fundamental Python syntax and semantics and be fluent in the use of Python control flow statements.
2. Develop proficiency in the handling of strings and functions.
3. Construct Python programs by utilizing the data structures like lists, dictionaries, tuples and sets.
4. Identify the commonly used operations involving file systems and exception handling.

CO–PO mapping as supplied

CO1: PO1 strong (3), PO4 strong (3).
CO2: PO1 strong (3), PO4 strong (3).
CO3: PO1 strong (3), PO4 strong (3).
CO4: PO1 strong (3), PO4 strong (3).
Legend: 3-Strongly mapped, 2-Moderately mapped, 1-Weakly mapped.

Complete Syllabus Coverage

Official syllabus point-by-point coverage

This audit layer maps every official source block to the corresponding lesson module. The source transcript is retained verbatim as extracted from the supplied syllabus; the study guidance below is educational support and is not presented as additional official syllabus content. No official point is intentionally omitted.

Source-to-lesson coverage map

Module	Lesson topic cards	Source basis	Official point units
Module 1	3	Official topic-row context	3
Module 2	3	Official topic-row context	3
Module 3	4	Official topic-row context	4
Module 4	3	Official topic-row context	3

Use this checklist to confirm that every official module topic is represented in the lesson.

Complete official topic coverage checklist

Module	Topic code	Topic	Hours
module-1	MO1.1	Operator precedence	2
module-1	MO1.2	Decision control structures	4
module-1	MO1.3	Loop control structures	4
module-2	MO2.1	Functions: swapping values and checking odd/even	3

Module	Topic code	Topic	Hours
module-2	MO2.3	Default arguments	3
module-2	MO2.3	Strings	6
module-3	MO3.1	Lists	3
module-3	MO3.2	Dictionaries	3
module-3	MO3.3	Lists: nested and practical operations	3
module-3	MO3.4	Sets	1
module-4	MO4.1	Exception handling	4
module-4	MO4.2	Create, open and close a file	3
module-4	MO4.3	Append a file and related functions	3

Learning Roadmap

1. Python Syntax, Operator Precedence and Control Flow
CO1 · 10 hours

2. Functions and Strings
CO2 · 12 hours

3. Lists, Dictionaries, Tuples and Sets
CO3 · 10 hours

4. Files and Exception Handling
CO4 · 10 hours

The roadmap follows the order of the supplied syllabus.

OFFICIAL MODULE 1

Python Syntax, Operator Precedence and Control Flow

This module establishes the grammar of a Python program and then uses decisions and repetition to express algorithms. The emphasis is on writing small, testable programs rather than memorising isolated syntax.

Hours: 10
CO1 · Bloom level: Applying

Handbook study routine: Complete Module 1 in three passes. First read the official source coverage and topic headings. Next study every Handbook treatment, writing the key definition, relationship, procedure, formula, diagram, or comparison in your own words. Finally close the notes and answer the self-check questions and the module questions without looking at the answer.

Official source coverage for Module 1

Source basis: Official topic-row context. Every point below is retained and linked to a study-oriented explanation. The main topic cards remain the primary lesson narrative.

View extracted official source transcript

MO1.1 2 Applying operator precedence Develop simple Python programs using decision MO1.2 4 Applying
MO1.2 4 Applying control structures Implement Python Programs using loop control MO1.3 4 Applying
MO1.3 4 Applying structures. CO2 Develop proficiency in the handling of strings and functions.

Point-by-point study guide

– Official syllabus point 1: MO1.1 2 Applying operator precedence Develop simple Python programs using decision MO1.2 4 Applying

Exact source point

> **Syllabus text:** MO1.1 2 Applying operator precedence Develop simple Python programs using decision MO1.2 4 Applying

How to understand and study it

This point is an official syllabus requirement: “MO1.1 2 Applying operator precedence Develop simple Python programs using decision MO1.2 4 Applying”. Study the input, logic or algorithm, output, and one test condition. Write the steps in order and identify the common error that changes the result. The main topic explanation above gives the concept context; this point-by-point section ensures that each named item is revised separately.

Study sequence

1. Write the exact technical term and a one-sentence definition in your own words.
2. Practise one small input-output example and test at least one boundary or fault condition.
3. Finish with one examination answer: definition, explanation, diagram/table if relevant, and application or limitation.

> **Malayalam support:** □ syllabus point □□□□□□□□□□□□ MO1.1 2 Applying operator precedence Develop simple Python programs using decision MO1.2 4 Applying □□□□ technical terms English-□□□□□□□ □□□□□□□. □□□□□ definition □□□□□□□□□□ principle □□□□□□□□□□□□□; □□□□ □□□ term-□□□□ function, difference, application □□□□□□ □□□□□□□□□□ □□□□□□. Diagram, sequence, formula, unit □□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□ revision □□□□□□□.

> **Exam focus:** Answer this point with its definition or principle first, followed by the named features, sequence, comparison, formula, diagram, or application that the question demands.

Handbook treatment

MO1.1 2 Applying operator precedence Develop simple Python programs using decision MO1.2 4 Applying should be understood as an information-flow problem. Identify the input, the rule or program that processes it, the internal state or decision, and the output. A correct explanation must show the sequence, not merely list software terms.

Learning objectives

- Define and scope MO1.1 2 Applying operator precedence Develop simple Python programs using decision MO1.2 4 Applying using the official terminology retained above.
- Organise the important elements of MO1.1 2 Applying operator precedence Develop simple Python programs using decision MO1.2 4 Applying into a logical sequence, classification, structure, or calculation.
- Connect MO1.1 2 Applying operator precedence Develop simple Python programs using decision MO1.2 4 Applying with one engineering decision, operation, measurement, or safety requirement in the context of Python Syntax, Operator Precedence and Control Flow.
- Write an examination answer on MO1.1 2 Applying operator precedence Develop simple Python programs using decision MO1.2 4 Applying with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading MO1.1 2 Applying operator precedence Develop simple Python programs using decision MO1.2 4 Applying. It is a study organisation method, not an additional official syllabus requirement.

- List the input signals, data, or conditions.
- Write the logic, algorithm, instruction sequence, or protocol rule in the order it is evaluated.
- State the output and identify what changes when an input or state changes.
- Test a normal case, a boundary case, and a fault or invalid-input case.

Engineering application lens

For engineering practice, MO1.1 2 Applying operator precedence Develop simple Python programs using decision MO1.2 4 Applying matters because an automated or digital system must convert inputs into repeatable outputs. The technician should be able to trace a signal or data item, locate the decision that controls the result, and identify a safe response when an input is missing or abnormal.

Worked answer method

Given: the input conditions, required output, and any stated constraints.
Required: the logic sequence, program structure, truth table, or operating result.
Method: break the problem into named states or steps; evaluate them in order; test at least one alternate condition.
Answer: show the final sequence and state the expected output for each important condition.

Exam answer blueprint

For a short-answer question on MO1.1 2 Applying operator precedence Develop simple Python programs using decision MO1.2 4 Applying, begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is MO1.1 2 Applying operator precedence Develop simple Python programs using decision MO1.2 4 Applying, and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining MO1.1 2 Applying operator precedence Develop simple Python programs using decision MO1.2 4 Applying?
- How would you distinguish MO1.1 2 Applying operator precedence Develop simple Python programs using decision MO1.2 4 Applying from a closely related idea?
- Where would an engineer or technician encounter MO1.1 2 Applying operator precedence Develop simple Python programs using decision MO1.2 4 Applying, and what must be checked before using it?
- What common mistake would make an answer or operation involving MO1.1 2 Applying operator precedence Develop simple Python programs using decision MO1.2 4 Applying incorrect?

Common mistakes and safety emphasis

- Writing instructions without defining the input and output.
- Ignoring scan order, state retention, initialization, or boundary conditions.
- Confusing a logical condition with the physical signal that represents it.
- Testing only the normal case and failing to consider a fault or interlock condition.

Malayalam support: MO1.1 2 Applying operator precedence Develop simple Python programs using decision MO1.2 4 Applying □□□□ topic □□□□□□□□□□□□□ □□□□□ exact technical term □□□□□□□□□□□□□□. □□□□ definition □□□□□□□□□□ principle, named parts/steps, application, limitation □□□□□□ □□□□□□□□ □□□□□□□□. English terminology, symbols, units, diagram labels □□□□□□ □□□□□□□□□. Exam answer □□□□□□□□□□ concept-□□□□ □□□□□–□□□ □□□□□ □□□□□□□□□□□□□□.

– Official syllabus point 2: MO1.2 4 Applying control structures Implement Python Programs using loop control MO1.3 4 Applying

Exact source point

> **Syllabus text:** MO1.2 4 Applying control structures Implement Python Programs using loop control MO1.3 4 Applying

How to understand and study it

This point is an official syllabus requirement: “MO1.2 4 Applying control structures Implement Python Programs using loop control MO1.3 4 Applying”. Study the input, logic or algorithm, output, and one test condition. Write the steps in order and identify the common error that changes the result. The main topic explanation above gives the concept context; this point-by-point section ensures that each named item is revised separately.

Study sequence

1. Write the exact technical term and a one-sentence definition in your own words.
2. Practise one small input-output example and test at least one boundary or fault condition.
3. Finish with one examination answer: definition, explanation, diagram/table if relevant, and application or limitation.

> **Malayalam support:** □ syllabus point □□□□□□□□□□□□ MO1.2 4 Applying control structures Implement Python Programs using loop control MO1.3 4 Applying □□□□ technical terms English-□□□□□□□ □□□□□□□. □□□□□ definition □□□□□□□□□□ principle □□□□□□□□□□□□□; □□□□ □□□ term-□□□□ function, difference, application □□□□□□ □□□□□□□□□□ □□□□□□. Diagram, sequence, formula, unit □□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□ revision □□□□□□□.

> **Exam focus:** Answer this point with its definition or principle first, followed by the named features, sequence, comparison, formula, diagram, or application that the question demands.

Handbook treatment

MO1.2 4 Applying control structures Implement Python Programs using loop control MO1.3 4 Applying should be understood as an information-flow problem. Identify the input, the rule or program that processes it, the internal state or decision, and the output. A correct explanation must show the sequence, not merely list software terms.

Learning objectives

- Define and scope MO1.2 4 Applying control structures Implement Python Programs using loop control MO1.3 4 Applying using the official terminology retained above.
- Organise the important elements of MO1.2 4 Applying control structures Implement Python Programs using loop control MO1.3 4 Applying into a logical sequence, classification, structure, or calculation.
- Connect MO1.2 4 Applying control structures Implement Python Programs using loop control MO1.3 4 Applying with one engineering decision, operation, measurement, or safety requirement in the context of Python Syntax, Operator Precedence and Control Flow.
- Write an examination answer on MO1.2 4 Applying control structures Implement Python Programs using loop control MO1.3 4 Applying with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading MO1.2 4 Applying control structures Implement Python Programs using loop control MO1.3 4 Applying. It is a study organisation method, not an additional official syllabus requirement.

- List the input signals, data, or conditions.
- Write the logic, algorithm, instruction sequence, or protocol rule in the order it is evaluated.
- State the output and identify what changes when an input or state changes.
- Test a normal case, a boundary case, and a fault or invalid-input case.

Engineering application lens

For engineering practice, MO1.2 4 Applying control structures Implement Python Programs using loop control MO1.3 4 Applying matters because an automated or digital system must convert inputs into repeatable outputs. The technician should be able to trace a signal or data item, locate the decision that controls the result, and identify a safe response when an input is missing or abnormal.

Worked answer method

Given: the input conditions, required output, and any stated constraints.
Required: the logic sequence, program structure, truth table, or operating result.
Method: break the problem into named states or steps; evaluate them in order; test at least one alternate condition.
Answer: show the final sequence and state the expected output for each important condition.

Exam answer blueprint

For a short-answer question on MO1.2 4 Applying control structures Implement Python Programs using loop control MO1.3 4 Applying, begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is MO1.2 4 Applying control structures Implement Python Programs using loop control MO1.3 4 Applying, and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining MO1.2 4 Applying control structures Implement Python Programs using loop control MO1.3 4 Applying?
- How would you distinguish MO1.2 4 Applying control structures Implement Python Programs using loop control MO1.3 4 Applying from a closely related idea?
- Where would an engineer or technician encounter MO1.2 4 Applying control structures Implement Python Programs using loop control MO1.3 4 Applying, and what must be checked before using it?
- What common mistake would make an answer or operation involving MO1.2 4 Applying control structures Implement Python Programs using loop control MO1.3 4 Applying incorrect?

Common mistakes and safety emphasis

- Writing instructions without defining the input and output.
- Ignoring scan order, state retention, initialization, or boundary conditions.
- Confusing a logical condition with the physical signal that represents it.
- Testing only the normal case and failing to consider a fault or interlock condition.

Malayalam support: MO1.2 4 Applying control structures Implement Python Programs using loop control MO1.3 4 Applying □□□□ topic □□□□□□□□□□□□□ □□□□□ exact technical term □□□□□□□□□□□□□□. □□□□ definition □□□□□□□□□□ principle, named parts/steps, application, limitation □□□□□□ □□□□□□□□ □□□□□□□□. English terminology, symbols, units, diagram labels □□□□□□ □□□□□□□□□. Exam answer □□□□□□□□□□ concept-□□□□ □□□□□–□□□ □□□□□ □□□□□□□□□□□□□□.

Official syllabus point 3: MO1.3 4 Applying structures. CO2 Develop proficiency in the handling of strings and functions.

Exact source point

> **Syllabus text:** MO1.3 4 Applying structures. CO2 Develop proficiency in the handling of strings and functions.

How to understand and study it

This point is an official syllabus requirement: “MO1.3 4 Applying structures. CO2 Develop proficiency in the handling of strings and functions.”. Treat every named term as an examinable subpoint. Define it, explain its role or behaviour, distinguish it from related terms, and connect it to the module application. The main topic explanation above gives the concept context; this point-by-point section ensures that each named item is revised separately.

Study sequence

1. Write the exact technical term and a one-sentence definition in your own words.
2. List the named terms separately and add one distinguishing feature or engineering use for each.
3. Finish with one examination answer: definition, explanation, diagram/table if relevant, and application or limitation.

> **Malayalam support:** □ syllabus point □□□□□□□□□□□□ MO1.3 4 Applying structures. CO2 Develop proficiency in the handling of strings, functions. □□□□ technical terms English-□□□□□□□ □□□□□□□. □□□□□ definition □□□□□□□□□□ principle □□□□□□□□□□□□□; □□□□ □□□ term-□□□□ function, difference, application □□□□□□ □□□□□□□□□□ □□□□□□. Diagram, sequence, formula, unit □□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□ revision □□□□□□□.

> **Exam focus:** Answer this point with its definition or principle first, followed by the named features, sequence, comparison, formula, diagram, or application that the question demands.

Handbook treatment

MO1.3 4 Applying structures. CO2 Develop proficiency in the handling of strings and functions. should be studied by linking structure, property, process, and application. The important question is why a material or process gives a particular behaviour and where that behaviour is useful or limiting.

Learning objectives

- Define and scope MO1.3 4 Applying structures. CO2 Develop proficiency in the handling of strings and functions. using the official terminology retained above.
- Organise the important elements of MO1.3 4 Applying structures. CO2 Develop proficiency in the handling of strings and functions. into a logical sequence, classification, structure, or calculation.
- Connect MO1.3 4 Applying structures. CO2 Develop proficiency in the handling of strings and functions. with one engineering decision, operation, measurement, or safety requirement in the context of Python Syntax, Operator Precedence and Control Flow.
- Write an examination answer on MO1.3 4 Applying structures. CO2 Develop proficiency in the handling of strings and functions. with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading MO1.3 4 Applying structures. CO2 Develop proficiency in the handling of strings and functions.. It is a study organisation method, not an additional official syllabus requirement.

- Identify the material, process, property, or defect named in the topic.
- Explain the relevant structure or mechanism at the level expected in the syllabus.
- Relate the mechanism to strength, hardness, conductivity, corrosion, manufacturability, durability, or another stated property.
- Finish with selection criteria, application, limitation, and safety or quality consideration.

Engineering application lens

In engineering practice, MO1.3 4 Applying structures. CO2 Develop proficiency in the handling of strings and functions. affects selection, manufacturing quality, service life, inspection, and maintenance. A technically useful answer links the observed property or defect to its cause and then to the method used to control or exploit it.

Worked answer method

Given: the material, process, property, or service condition in the question.
Required: explanation, comparison, selection, or identification of the relevant mechanism.
Method: state the principle; connect cause to property; compare alternatives using the same criteria; mention the relevant test or control.
Answer: give the conclusion with the application and the principal limitation.

Exam answer blueprint

For a short-answer question on MO1.3 4 Applying structures. CO2 Develop proficiency in the handling of strings and functions., begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is MO1.3 4 Applying structures. CO2 Develop proficiency in the handling of strings and functions., and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining MO1.3 4 Applying structures. CO2 Develop proficiency in the handling of strings and functions.?
- How would you distinguish MO1.3 4 Applying structures. CO2 Develop proficiency in the handling of strings and functions. from a closely related idea?
- Where would an engineer or technician encounter MO1.3 4 Applying structures. CO2 Develop proficiency in the handling of strings and functions., and what must be checked before using it?
- What common mistake would make an answer or operation involving MO1.3 4 Applying structures. CO2 Develop proficiency in the handling of strings and functions. incorrect?

Common mistakes and safety emphasis

- Memorising a property without its unit, condition, or engineering meaning.
- Confusing a manufacturing process with a material property.
- Giving a comparison with different criteria for different materials.
- Ignoring defects, surface condition, heat treatment, environment, or quality control.

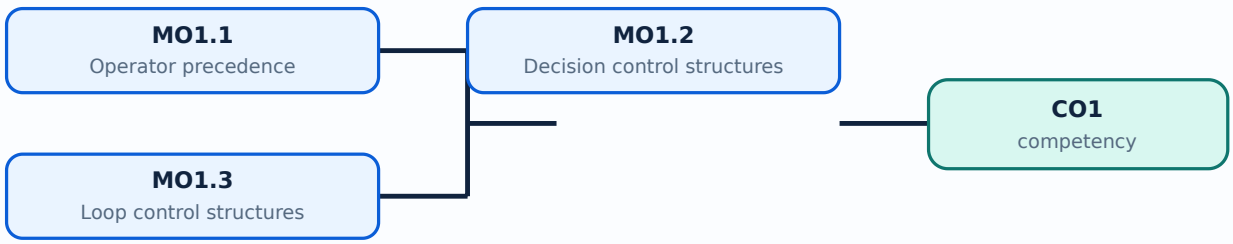
Malayalam support: MO1.3 4 Applying structures. CO2 Develop proficiency in the handling of strings and functions. □□□□ topic □□□□□□□□□□□□□ □□□□□ exact technical term □□□□□□□□□□□□□□. □□□□ definition □□□□□□□□□□ principle, named parts/steps, application, limitation □□□□□□ □□□□□□□□ □□□□□□□□. English terminology, symbols, units, diagram labels □□□□□□ □□□□□□□□□. Exam answer □□□□□□□□□□ concept-□□□□ □□□□□–□□□ □□□□□ □□□□□□□□□□□□□□.

Learning goals

- Recognise literals, variables, expressions, indentation, input/output, and operator precedence.
- Select an appropriate decision structure for a problem.
- Use for and while loops with correct initialisation, condition, update, and termination.

Module study tip

Read every topic in sequence, reproduce its example or procedure, and write a short explanation before attempting the module questions.



Learning map for Module module-1: the listed topics build the module competency.

MO1.1 — Operator precedence (2 hours)

Concept and explanation

Python evaluates an expression according to a precedence hierarchy. Parentheses are evaluated first, followed by exponentiation, unary operators, multiplication/division/floor division/modulus, addition/subtraction, comparisons, membership or identity tests, and Boolean not, and, or. When an expression is not immediately clear, parentheses are the safest way to communicate the intended order. Assignment is a statement rather than an arithmetic operator.

Malayalam support: Python-□ expression evaluate □□□□□□□□□□□ operator precedence □□□ order □□□□□□□□□□□□□□□□□□. □□□□□ □□□□□□□□□□ brackets □□□□□□□□□□□□□□□ □□□□□□□ □□□□□□□□□□□□.

Study points

- Use meaningful variable names; distinguish / from //; test expressions with small values; avoid depending on remembered precedence for long expressions.

Engineering / workplace application: Operator precedence is important in marks calculation, sensor thresholds, billing expressions, and validation logic.

Illustrative example: Given a=8 and b=3, `a + b * 2` gives 14 because multiplication is evaluated first; `(a+b)*2` gives 22.

Common mistake: Writing a*b+c as a*(b+c) without checking the intended formula changes the result.

Handbook treatment

MO1.1 — Operator precedence (2 hours) is an official examinable part of the module. Study it as a connected explanation: define the central term, identify the related elements, explain their relationship, and finish with a relevant engineering use or limitation.

Learning objectives

- Define and scope MO1.1 — Operator precedence (2 hours) using the official terminology retained above.
- Organise the important elements of MO1.1 — Operator precedence (2 hours) into a logical sequence, classification, structure, or calculation.
- Connect MO1.1 — Operator precedence (2 hours) with one engineering decision, operation, measurement, or safety requirement in the context of Python Syntax, Operator Precedence and Control Flow.
- Write an examination answer on MO1.1 — Operator precedence (2 hours) with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading MO1.1 — Operator precedence (2 hours). It is a study organisation method, not an additional official syllabus requirement.

- Start with the exact technical term and its scope.
- Separate the topic into named elements, stages, types, or conditions.
- Explain the relationship using cause-and-effect, sequence, comparison, or a labelled representation.
- Apply the idea to a realistic engineering situation and state one limitation or common mistake.

Engineering application lens

The engineering value of MO1.1 — Operator precedence (2 hours) is understood by asking what requirement it satisfies, what input or condition it depends on, how the output is obtained, and how a technician or designer verifies the result. Use the module context to make the application specific rather than memorising an isolated definition.

Worked answer method

Given: the topic, operating condition, diagram, data, or situation stated in the question.
Required: definition, explanation, comparison, procedure, calculation, or application as requested.
Method: organise the answer under principle, named elements, sequence or relationship, and application.
Answer: use the requested technical terms, units, labels, assumptions, and a concluding statement.

Exam answer blueprint

For a short-answer question on MO1.1 — Operator precedence (2 hours), begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is MO1.1 — Operator precedence (2 hours), and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining MO1.1 — Operator precedence (2 hours)?
- How would you distinguish MO1.1 — Operator precedence (2 hours) from a closely related idea?
- Where would an engineer or technician encounter MO1.1 — Operator precedence (2 hours), and what must be checked before using it?
- What common mistake would make an answer or operation involving MO1.1 — Operator precedence (2 hours) incorrect?

Common mistakes and safety emphasis

- Copying a definition without explaining the mechanism or purpose.
- Listing terms without showing their relationship.
- Using an example that does not match the stated topic or condition.
- Leaving out a diagram, unit, assumption, limitation, or safety point when the question requires it.

Malayalam support: MO1.1 — Operator precedence (2 hours) □□□□ topic □□□□□□□□□□□□□ □□□□□□ exact technical term □□□□□□□□□□□□□□□. □□□□□ definition □□□□□□□□□□□ principle, named parts/steps, application, limitation □□□□□□□ □□□□□□□□□ □□□□□□□□□. English terminology, symbols, units, diagram labels □□□□□□□ □□□□□□□□□□. Exam answer □□□□□□□□□□□ concept-□□□□□ □□□□□□–□□□ □□□□□□ □□□□□□□□□□□□□□□.

MO1.2 — Decision control structures (4 hours)

Concept and explanation

Decision structures choose a path based on a Boolean condition. Python uses if, elif, and else with indentation to delimit blocks. Conditions may combine comparisons with and, or, and not; nested decisions should be used only when they make the logic clearer. A complete program should consider equality, range boundaries, and invalid input.

Malayalam support: Condition □□□□□□□□ □□□□□□□□□ □□□□□□□□□ □□□□□□□□□□□□□□□ program □□□ □□□ □□□□□□ □□□□□□ if/elif/else □□□□□□□□□□□□□□□□□□. Indentation Python-□ block □□□□□□□□□□□□□□□□□ □□□□□□□□□ □□□□□□□□□□□.

Study points

- Convert the problem statement into conditions; put mutually exclusive cases in a clear order; include an else or validation branch where appropriate; test boundary values.

Engineering / workplace application: Grade classification, tax slabs, pass/fail decisions, menu selection, and machine interlocks use decision logic.

Illustrative example: For marks m, `if m>=75: distinction; elif m>=50: pass; else: fail` evaluates the highest applicable band first.

Common mistake: Using `if x > 10` when the requirement is `x >= 10` loses the boundary case.

Handbook treatment

MO1.2 — Decision control structures (4 hours) should be learned through the chain of measurand, sensing element, signal conversion, indication or processing, and decision. The purpose of every stage is more important than memorising a component name alone.

Learning objectives

- Define and scope MO1.2 — Decision control structures (4 hours) using the official terminology retained above.
- Organise the important elements of MO1.2 — Decision control structures (4 hours) into a logical sequence, classification, structure, or calculation.
- Connect MO1.2 — Decision control structures (4 hours) with one engineering decision, operation, measurement, or safety requirement in the context of Python Syntax, Operator Precedence and Control Flow.
- Write an examination answer on MO1.2 — Decision control structures (4 hours) with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading MO1.2 — Decision control structures (4 hours). It is a study organisation method, not an additional official syllabus requirement.

- Define the physical quantity or measurand.
- Identify the sensing or conversion element and the form of output it produces.
- Explain conditioning, transmission, indication, or control if present.
- Discuss range, sensitivity, accuracy, repeatability, response, calibration, and limitations where relevant.

Engineering application lens

Engineering use of MO1.2 — Decision control structures (4 hours) depends on whether the measurement is sufficiently accurate, fast, repeatable, robust, and compatible with the controller or display. The selection must also consider installation, environment, maintenance, and failure mode.

Worked answer method

Given: the quantity to be sensed, the operating environment, and the required decision or control action.
Required: a suitable measurement chain or an explanation of how the instrument operates.
Method: map measurand → sensing element → signal → processing/indication → action; identify one source of error and one calibration check.
Answer: state the measured quantity, principle, important characteristics, application, and limitation.

Exam answer blueprint

For a short-answer question on MO1.2 — Decision control structures (4 hours), begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is MO1.2 — Decision control structures (4 hours), and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining MO1.2 — Decision control structures (4 hours)?
- How would you distinguish MO1.2 — Decision control structures (4 hours) from a closely related idea?
- Where would an engineer or technician encounter MO1.2 — Decision control structures (4 hours), and what must be checked before using it?
- What common mistake would make an answer or operation involving MO1.2 — Decision control structures (4 hours) incorrect?

Common mistakes and safety emphasis

- Confusing accuracy, precision, sensitivity, and resolution.
- Calling every electrical output a direct measurement without explaining conversion.
- Ignoring calibration, loading, response time, or environmental effects.
- Failing to state the unit and reference condition of the measurand.

Malayalam support: MO1.2 — Decision control structures (4 hours) □□□□ topic □□□□□□□□□□□□□ □□□□□□ exact technical term □□□□□□□□□□□□□□□. □□□□□ definition □□□□□□□□□□□ principle, named parts/steps, application, limitation □□□□□□□ □□□□□□□□□ □□□□□□□□□. English terminology, symbols, units, diagram labels □□□□□□□ □□□□□□□□□□. Exam answer □□□□□□□□□□□ concept-□□□□□ □□□□□□–□□□ □□□□□□ □□□□□□□□□□□□□□□.

MO1.3 — Loop control structures (4 hours)

Concept and explanation

A loop repeats a block while a pattern or condition remains valid. A for loop is suitable for iterating through a known sequence or range; a while loop is suitable when repetition depends on a changing condition. Initialise the control variable, update it, and make the exit condition reachable. break stops a loop, continue skips to the next iteration, and an else clause can run when a loop ends normally.

Malayalam support: Loop □□□□□□□□□□ □□□ □□□□□ □□ □□□ □□□□□□□□□□□□. for □□□□□□□□□□ sequence-□□□□□□□□□ □□□□□□□□□□□; while condition □□□□□□□□□□□□ □□□□□□□□□□□□□□. Exit condition □□□□□□□□□□ infinite loop □□□□□□□□.

Study points

- Trace one iteration in a table; keep loop bodies short; use range endpoints carefully; prevent infinite loops; validate input before repeating.

Engineering / workplace application: Counters, data sampling, menu-driven programs, and repeated file records are natural loop applications.

Illustrative example: A loop from 1 to 5 inclusive is `for i in range(1, 6):`; the stop value in range is excluded.

Common mistake: Forgetting to update a while-loop variable causes an infinite loop; using range(1,5) when 5 must be included also causes an error.

Handbook treatment

MO1.3 — Loop control structures (4 hours) should be learned through the chain of measurand, sensing element, signal conversion, indication or processing, and decision. The purpose of every stage is more important than memorising a component name alone.

Learning objectives

- Define and scope MO1.3 — Loop control structures (4 hours) using the official terminology retained above.
- Organise the important elements of MO1.3 — Loop control structures (4 hours) into a logical sequence, classification, structure, or calculation.
- Connect MO1.3 — Loop control structures (4 hours) with one engineering decision, operation, measurement, or safety requirement in the context of Python Syntax, Operator Precedence and Control Flow.
- Write an examination answer on MO1.3 — Loop control structures (4 hours) with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading MO1.3 — Loop control structures (4 hours). It is a study organisation method, not an additional official syllabus requirement.

- Define the physical quantity or measurand.
- Identify the sensing or conversion element and the form of output it produces.
- Explain conditioning, transmission, indication, or control if present.
- Discuss range, sensitivity, accuracy, repeatability, response, calibration, and limitations where relevant.

Engineering application lens

Engineering use of MO1.3 — Loop control structures (4 hours) depends on whether the measurement is sufficiently accurate, fast, repeatable, robust, and compatible with the controller or display. The selection must also consider installation, environment, maintenance, and failure mode.

Worked answer method

Given: the quantity to be sensed, the operating environment, and the required decision or control action.
Required: a suitable measurement chain or an explanation of how the instrument operates.
Method: map measurand → sensing element → signal → processing/indication → action; identify one source of error and one calibration check.
Answer: state the measured quantity, principle, important characteristics, application, and limitation.

Exam answer blueprint

For a short-answer question on MO1.3 — Loop control structures (4 hours), begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is MO1.3 — Loop control structures (4 hours), and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining MO1.3 — Loop control structures (4 hours)?
- How would you distinguish MO1.3 — Loop control structures (4 hours) from a closely related idea?
- Where would an engineer or technician encounter MO1.3 — Loop control structures (4 hours), and what must be checked before using it?
- What common mistake would make an answer or operation involving MO1.3 — Loop control structures (4 hours) incorrect?

Common mistakes and safety emphasis

- Confusing accuracy, precision, sensitivity, and resolution.
- Calling every electrical output a direct measurement without explaining conversion.
- Ignoring calibration, loading, response time, or environmental effects.
- Failing to state the unit and reference condition of the measurand.

Malayalam support: MO1.3 — Loop control structures (4 hours) □□□□ topic □□□□□□□□□□□□□ □□□□□ exact technical term □□□□□□□□□□□□□□. □□□□ definition □□□□□□□□□□ principle, named parts/steps, application, limitation □□□□□□ □□□□□□□□ □□□□□□□□. English terminology, symbols, units, diagram labels □□□□□□ □□□□□□□□□□. Exam answer □□□□□□□□□□ concept-□□□□ □□□□□–□□□ □□□□□ □□□□□□□□□□□□□□.

Module summary: Syntax supports expressions; decisions select one path; loops express controlled repetition.

OFFICIAL MODULE 2

Functions and Strings

Functions divide a solution into named, reusable units. Strings provide a safe way to represent and process text; together they support readable programs for validation, searching, formatting, and small automation tasks.

Hours: 12
CO2 · Bloom level: Applying

Handbook study routine: Complete Module 2 in three passes. First read the official source coverage and topic headings. Next study every Handbook treatment, writing the key definition, relationship, procedure, formula, diagram, or comparison in your own words. Finally close the notes and answer the self-check questions and the module questions without looking at the answer.

Official source coverage for Module 2

Source basis: Official topic-row context. Every point below is retained and linked to a study-oriented explanation. The main topic cards remain the primary lesson narrative.

View extracted official source transcript

MO2.1 swapping the value in two variables, to check 3 Applying whether a given number is odd or even, etc) MO2.3 Python program to demonstrate default arguments 3 Applying
MO2.3 Python program to demonstrate default arguments 3 Applying MO2.3 Implement Python programs using strings 6 Applying
MO2.3 Implement Python programs using strings 6 Applying Lab Exam – I 1½

Point-by-point study guide

– Official syllabus point 1: MO2.1 swapping the value in two variables, to check 3 Applying whether a given number is odd or even, etc) MO2.3 Python program to demonstrate default arguments 3 Applying

Exact source point

> **Syllabus text:** MO2.1 swapping the value in two variables, to check 3 Applying whether a given number is odd or even, etc) MO2.3 Python program to demonstrate default arguments 3 Applying

How to understand and study it

This point is an official syllabus requirement: “MO2.1 swapping the value in two variables, to check 3 Applying whether a given number is odd or even, etc) MO2.3 Python program to demonstrate default arguments 3 Applying”. Record the relevant quantity, symbol, unit, relation or formula, and the interpretation of the result. Do not memorise a number without its unit and condition. The main topic explanation above gives the concept context; this point-by-point section ensures that each named item is revised separately.

Study sequence

1. Write the exact technical term and a one-sentence definition in your own words.
2. Practise one small input-output example and test at least one boundary or fault condition.
3. Finish with one examination answer: definition, explanation, diagram/table if relevant, and application or limitation.

> **Malayalam support:** □ syllabus point □□□□□□□□□□□□ MO2.1 swapping the value in two variables, to check 3 Applying whether a given number is odd or even, etc) MO2.3 Python program to demonstrate default arguments 3 Applying □□□□ technical terms English-□□□□□□□ □□□□□□□. □□□□□ definition □□□□□□□□□□ principle □□□□□□□□□□□□□; □□□□ □□□ term-□□□□ function, difference, application □□□□□□ □□□□□□□□□□ □□□□□□. Diagram, sequence, formula, unit □□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□ revision □□□□□□□.

> **Exam focus:** Answer this point with its definition or principle first, followed by the named features, sequence, comparison, formula, diagram, or application that the question demands.

Handbook treatment

MO2.1 swapping the value in two variables, to check 3 Applying whether a given number is odd or even, etc) MO2.3 Python program to demonstrate default arguments 3 Applying must be learned as both a concept and a calculation procedure. The student should know what each quantity represents, the conditions under which a relation is valid, the unit of every quantity, and how to check whether the final answer is physically reasonable.

Learning objectives

- Define and scope MO2.1 swapping the value in two variables, to check 3 Applying whether a given number is odd or even, etc) MO2.3 Python program to demonstrate default arguments 3 Applying using the official terminology retained above.
- Organise the important elements of MO2.1 swapping the value in two variables, to check 3 Applying whether a given number is odd or even, etc) MO2.3 Python program to demonstrate default arguments 3 Applying into a logical sequence, classification, structure, or calculation.
- Connect MO2.1 swapping the value in two variables, to check 3 Applying whether a given number is odd or even, etc) MO2.3 Python program to demonstrate default arguments 3 Applying with one engineering decision, operation, measurement, or safety requirement in the context of Functions and Strings.
- Write an examination answer on MO2.1 swapping the value in two variables, to check 3 Applying whether a given number is odd or even, etc) MO2.3 Python program to demonstrate default arguments 3 Applying with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading MO2.1 swapping the value in two variables, to check 3 Applying whether a given number is odd or even, etc) MO2.3 Python program to demonstrate default arguments 3 Applying. It is a study organisation method, not an additional official syllabus requirement.

- Identify the given quantities and write them with symbols and SI units.
- Select the governing definition, equation, graph, or relationship instead of starting with arithmetic.
- Substitute values only after checking units and the stated operating condition.
- Interpret the result in words and check its sign, magnitude, unit, and limiting behaviour.

Engineering application lens

In an engineering setting, MO2.1 swapping the value in two variables, to check 3 Applying whether a given number is odd or even, etc) MO2.3 Python program to demonstrate default arguments 3 Applying is useful when a designer or technician must convert an observation into a measurable value, predict a response, compare alternatives, or verify that a component is operating within its rated condition. The practical question is always: what is measured or known, what is required, what model is appropriate, and how will the result be checked?

Worked answer method

Given: the quantities and conditions stated in the question.
Required: the unknown quantity, including its unit.
Calculation: write the governing relation symbolically, substitute consistently converted values, and show the unit cancellation.
Answer: report the result with a suitable number of significant figures and one sentence of interpretation.

Exam answer blueprint

For a short-answer question on MO2.1 swapping the value in two variables, to check 3 Applying whether a given number is odd or even, etc) MO2.3 Python program to demonstrate default arguments 3 Applying, begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is MO2.1 swapping the value in two variables, to check 3 Applying whether a given number is odd or even, etc) MO2.3 Python program to demonstrate default arguments 3 Applying, and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining MO2.1 swapping the value in two variables, to check 3 Applying whether a given number is odd or even, etc) MO2.3 Python program to demonstrate default arguments 3 Applying?
- How would you distinguish MO2.1 swapping the value in two variables, to check 3 Applying whether a given number is odd or even, etc) MO2.3 Python program to demonstrate default arguments 3 Applying from a closely related idea?
- Where would an engineer or technician encounter MO2.1 swapping the value in two variables, to check 3 Applying whether a given number is odd or even,

etc) MO2.3 Python program to demonstrate default arguments 3 Applying, and what must be checked before using it?

- What common mistake would make an answer or operation involving MO2.1 swapping the value in two variables, to check 3 Applying whether a given number is odd or even, etc) MO2.3 Python program to demonstrate default arguments 3 Applying incorrect?

Common mistakes and safety emphasis

- Using a memorised relation without checking its conditions.
- Mixing prefixes or units, such as milli, kilo, mega, seconds, minutes, or degrees.
- Reporting a numerical value without a unit or without explaining what it means.
- Rounding intermediate values so aggressively that the final answer changes.

Malayalam support: MO2.1 swapping the value in two variables, to check 3 Applying whether a given number is odd or even, etc) MO2.3 Python program to demonstrate default arguments 3 Applying □□□□ topic □□□□□□□□□□□□ □□□□□ exact technical term □□□□□□□□□□□□□. □□□□ definition □□□□□□□□□□ principle, named parts/steps, application, limitation □□□□□□ □□□□□□□□ □□□□□□□□. English terminology, symbols, units, diagram labels □□□□□□ □□□□□□□□□. Exam answer □□□□□□□□□□ concept-□□□□ □□□□□–□□□ □□□□□ □□□□□□□□□□□□□.

– Official syllabus point 2: MO2.3 Python program to demonstrate default arguments 3 Applying MO2.3 Implement Python programs using strings 6 Applying

Exact source point

> **Syllabus text:** MO2.3 Python program to demonstrate default arguments 3 Applying MO2.3 Implement Python programs using strings 6 Applying

How to understand and study it

This point is an official syllabus requirement: “MO2.3 Python program to demonstrate default arguments 3 Applying MO2.3 Implement Python programs using strings 6 Applying”. Study the input, logic or algorithm, output, and one test condition. Write the steps in order and identify the common error that changes the result. The main topic explanation above gives the concept context; this point-by-point section ensures that each named item is revised separately.

Study sequence

1. Write the exact technical term and a one-sentence definition in your own words.
2. Practise one small input-output example and test at least one boundary or fault condition.
3. Finish with one examination answer: definition, explanation, diagram/table if relevant, and application or limitation.

> **Malayalam support:** □ syllabus point □□□□□□□□□□□□ MO2.3 Python program to demonstrate default arguments 3 Applying MO2.3 Implement Python programs using strings 6 Applying □□□□ technical terms English-□□□□□□□ □□□□□□□. □□□□□ definition □□□□□□□□□□ principle □□□□□□□□□□□□□; □□□□ □□□ term-□□□□ function, difference, application □□□□□□ □□□□□□□□□□ □□□□□□. Diagram, sequence, formula, unit □□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□ revision □□□□□□□.

> **Exam focus:** Answer this point with its definition or principle first, followed by the named features, sequence, comparison, formula, diagram, or application that the question demands.

Handbook treatment

MO2.3 Python program to demonstrate default arguments 3 Applying MO2.3 Implement Python programs using strings 6 Applying should be understood as an information-flow problem. Identify the input, the rule or program that processes it, the internal state or decision, and the output. A correct explanation must show the sequence, not merely list software terms.

Learning objectives

- Define and scope MO2.3 Python program to demonstrate default arguments 3 Applying MO2.3 Implement Python programs using strings 6 Applying using the official terminology retained above.
- Organise the important elements of MO2.3 Python program to demonstrate default arguments 3 Applying MO2.3 Implement Python programs using strings 6 Applying into a logical sequence, classification, structure, or calculation.
- Connect MO2.3 Python program to demonstrate default arguments 3 Applying MO2.3 Implement Python programs using strings 6 Applying with one engineering decision, operation, measurement, or safety requirement in the context of Functions and Strings.
- Write an examination answer on MO2.3 Python program to demonstrate default arguments 3 Applying MO2.3 Implement Python programs using strings 6 Applying with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading MO2.3 Python program to demonstrate default arguments 3 Applying MO2.3 Implement Python programs using strings 6 Applying. It is a study organisation method, not an additional official syllabus requirement.

- List the input signals, data, or conditions.
- Write the logic, algorithm, instruction sequence, or protocol rule in the order it is evaluated.
- State the output and identify what changes when an input or state changes.
- Test a normal case, a boundary case, and a fault or invalid-input case.

Engineering application lens

For engineering practice, MO2.3 Python program to demonstrate default arguments 3 Applying MO2.3 Implement Python programs using strings 6 Applying matters because an automated or digital system must convert inputs into repeatable outputs. The technician should be able to trace a signal or data

item, locate the decision that controls the result, and identify a safe response when an input is missing or abnormal.

Worked answer method

Given: the input conditions, required output, and any stated constraints.
Required: the logic sequence, program structure, truth table, or operating result.
Method: break the problem into named states or steps; evaluate them in order; test at least one alternate condition.
Answer: show the final sequence and state the expected output for each important condition.

Exam answer blueprint

For a short-answer question on MO2.3 Python program to demonstrate default arguments 3 Applying MO2.3 Implement Python programs using strings 6 Applying, begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is MO2.3 Python program to demonstrate default arguments 3 Applying MO2.3 Implement Python programs using strings 6 Applying, and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining MO2.3 Python program to demonstrate default arguments 3 Applying MO2.3 Implement Python programs using strings 6 Applying?
- How would you distinguish MO2.3 Python program to demonstrate default arguments 3 Applying MO2.3 Implement Python programs using strings 6 Applying from a closely related idea?
- Where would an engineer or technician encounter MO2.3 Python program to demonstrate default arguments 3 Applying MO2.3 Implement Python programs using strings 6 Applying, and what must be checked before using it?
- What common mistake would make an answer or operation involving MO2.3 Python program to demonstrate default arguments 3 Applying MO2.3 Implement Python programs using strings 6 Applying incorrect?

Common mistakes and safety emphasis

- Writing instructions without defining the input and output.
- Ignoring scan order, state retention, initialization, or boundary conditions.
- Confusing a logical condition with the physical signal that represents it.
- Testing only the normal case and failing to consider a fault or interlock condition.

Malayalam support: MO2.3 Python program to demonstrate default arguments 3 Applying MO2.3 Implement Python programs using strings 6 Applying □□□□ topic □□□□□□□□□□□□ □□□□□ exact technical term □□□□□□□□□□□□□. □□□□ definition □□□□□□□□□□ principle, named parts/steps, application, limitation □□□□□□ □□□□□□□□ □□□□□□□□. English terminology, symbols, units, diagram labels □□□□□□ □□□□□□□□□. Exam answer □□□□□□□□□□ concept-□□□□ □□□□□–□□□ □□□□□ □□□□□□□□□□□□□.

– Official syllabus point 3: MO2.3 Implement Python programs using strings 6 Applying Lab Exam – I 1½

Exact source point

> **Syllabus text:** MO2.3 Implement Python programs using strings 6 Applying Lab Exam – I 1½

How to understand and study it

This point is an official syllabus requirement: “MO2.3 Implement Python programs using strings 6 Applying Lab Exam – I 1½”. Study the input, logic or algorithm, output, and one test condition. Write the steps in order and identify the common error that changes the result. The main topic explanation above gives the concept context; this point-by-point section ensures that each named item is revised separately.

Study sequence

1. Write the exact technical term and a one-sentence definition in your own words.
2. Practise one small input-output example and test at least one boundary or fault condition.
3. Finish with one examination answer: definition, explanation, diagram/table if relevant, and application or limitation.

> **Malayalam support:** □ syllabus point □□□□□□□□□□□□ MO2.3 Implement Python programs using strings 6 Applying Lab Exam – I 1½ □□□□ technical terms English-□□□□□□□ □□□□□□□. □□□□□ definition □□□□□□□□□□ principle □□□□□□□□□□□□□; □□□□ □□□ term-□□□□ function, difference, application □□□□□□ □□□□□□□□□□ □□□□□□. Diagram, sequence, formula, unit □□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□ revision □□□□□□□.

> **Exam focus:** Answer this point with its definition or principle first, followed by the named features, sequence, comparison, formula, diagram, or application that the question demands.

Handbook treatment

MO2.3 Implement Python programs using strings 6 Applying Lab Exam – I 1½ should be understood as an information-flow problem. Identify the input, the rule or program that processes it, the internal state or decision, and the output. A correct explanation must show the sequence, not merely list software terms.

Learning objectives

- Define and scope MO2.3 Implement Python programs using strings 6 Applying Lab Exam – I 1½ using the official terminology retained above.
- Organise the important elements of MO2.3 Implement Python programs using strings 6 Applying Lab Exam – I 1½ into a logical sequence, classification, structure, or calculation.
- Connect MO2.3 Implement Python programs using strings 6 Applying Lab Exam – I 1½ with one engineering decision, operation, measurement, or safety requirement in the context of Functions and Strings.
- Write an examination answer on MO2.3 Implement Python programs using strings 6 Applying Lab Exam – I 1½ with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading MO2.3 Implement Python programs using strings 6 Applying Lab Exam – I 1½. It is a study organisation method, not an additional official syllabus requirement.

- List the input signals, data, or conditions.
- Write the logic, algorithm, instruction sequence, or protocol rule in the order it is evaluated.
- State the output and identify what changes when an input or state changes.
- Test a normal case, a boundary case, and a fault or invalid-input case.

Engineering application lens

For engineering practice, MO2.3 Implement Python programs using strings 6 Applying Lab Exam – I 1½ matters because an automated or digital system must convert inputs into repeatable outputs. The technician should be able to trace a signal or data item, locate the decision that controls the result, and identify a safe response when an input is missing or abnormal.

Worked answer method

Given: the input conditions, required output, and any stated constraints.
Required: the logic sequence, program structure, truth table, or operating result.
Method: break the problem into named states or steps; evaluate them in order; test at least one alternate condition.
Answer: show the final sequence and state the expected output for each important condition.

Exam answer blueprint

For a short-answer question on MO2.3 Implement Python programs using strings 6 Applying Lab Exam – I 1½, begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is MO2.3 Implement Python programs using strings 6 Applying Lab Exam – I 1½, and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining MO2.3 Implement Python programs using strings 6 Applying Lab Exam – I 1½?
- How would you distinguish MO2.3 Implement Python programs using strings 6 Applying Lab Exam – I 1½ from a closely related idea?
- Where would an engineer or technician encounter MO2.3 Implement Python programs using strings 6 Applying Lab Exam – I 1½, and what must be checked before using it?
- What common mistake would make an answer or operation involving MO2.3 Implement Python programs using strings 6 Applying Lab Exam – I 1½ incorrect?

Common mistakes and safety emphasis

- Writing instructions without defining the input and output.
- Ignoring scan order, state retention, initialization, or boundary conditions.
- Confusing a logical condition with the physical signal that represents it.
- Testing only the normal case and failing to consider a fault or interlock condition.

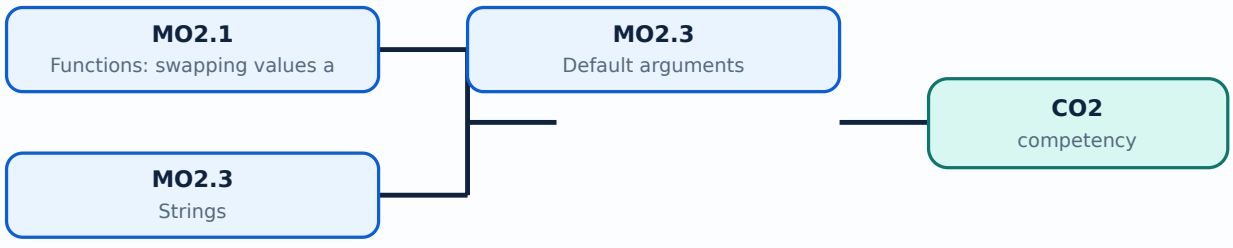
Malayalam support: MO2.3 Implement Python programs using strings 6 Applying Lab Exam – I 1½ □□□□ topic □□□□□□□□□□□□ □□□□□ exact technical term □□□□□□□□□□□□□. □□□□ definition □□□□□□□□□□ principle, named parts/steps, application, limitation □□□□□□ □□□□□□□□ □□□□□□□□. English terminology, symbols, units, diagram labels □□□□□□ □□□□□□□□□. Exam answer □□□□□□□□□□ concept-□□□□ □□□□□–□□□ □□□□□ □□□□□□□□□□□□□□.

Learning goals

- Define and call functions with parameters and return values.
- Use default arguments and understand local scope.
- Apply common string operations without confusing indexing, slicing, and mutation.

Module study tip

Read every topic in sequence, reproduce its example or procedure, and write a short explanation before attempting the module questions.



Learning map for Module module-2: the listed topics build the module competency.

MO2.1 — Functions: swapping values and checking odd/even (3 hours)

Concept and explanation

A function is defined with `def`, accepts parameters, performs a focused task, and may return a result. In Python, assignment can swap two variables as `a, b = b, a`, without a temporary variable. An odd/even function can use the remainder operator: n % 2 is zero for an even integer. The function should be tested with positive, negative, and zero inputs.

Malayalam support: Function □□□□□□ □□□ □□□□ □□□□□□ □□□□□□□□□□ reusable code block □□□. `a,b=b,a` □□□□□□□□□□ variables swap □□□□□□□. `% 2` □□□□□□□□□□ number odd □□□ even □□□ □□□□□□□□□□□□.

Study points

- Keep one responsibility per function; choose descriptive names; return values instead of printing when the caller may need the result; document assumptions.

Engineering / workplace application: Functions are used for reusable validation, payroll rules, conversion routines, and laboratory data processing.

Illustrative example: `def is_even(n): return n % 2 == 0` returns a Boolean value that can be used by another function.

Common mistake: Printing inside a function when the caller expects a returned value makes composition difficult.

Handbook treatment

MO2.1 — Functions: swapping values and checking odd/even (3 hours) must be learned as both a concept and a calculation procedure. The student should know what each quantity represents, the conditions under which a relation is valid, the unit of every quantity, and how to check whether the final answer is physically reasonable.

Learning objectives

- Define and scope MO2.1 — Functions: swapping values and checking odd/even (3 hours) using the official terminology retained above.
- Organise the important elements of MO2.1 — Functions: swapping values and checking odd/even (3 hours) into a logical sequence, classification, structure, or calculation.
- Connect MO2.1 — Functions: swapping values and checking odd/even (3 hours) with one engineering decision, operation, measurement, or safety requirement in the context of Functions and Strings.
- Write an examination answer on MO2.1 — Functions: swapping values and checking odd/even (3 hours) with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading MO2.1 — Functions: swapping values and checking odd/even (3 hours). It is a study organisation method, not an additional official syllabus requirement.

- Identify the given quantities and write them with symbols and SI units.
- Select the governing definition, equation, graph, or relationship instead of starting with arithmetic.
- Substitute values only after checking units and the stated operating condition.
- Interpret the result in words and check its sign, magnitude, unit, and limiting behaviour.

Engineering application lens

In an engineering setting, MO2.1 — Functions: swapping values and checking odd/even (3 hours) is useful when a designer or technician must convert an observation into a measurable value, predict a response, compare alternatives, or verify that a component is operating within its rated condition. The practical question is always: what is measured or known, what is required, what model is appropriate, and how will the result be checked?

Worked answer method

Given: the quantities and conditions stated in the question.
Required: the unknown quantity, including its unit.
Calculation: write the governing relation symbolically, substitute consistently converted values, and show the unit cancellation.
Answer: report the result with a suitable number of significant figures and one sentence of interpretation.

Exam answer blueprint

For a short-answer question on MO2.1 — Functions: swapping values and checking odd/even (3 hours), begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is MO2.1 — Functions: swapping values and checking odd/even (3 hours), and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining MO2.1 — Functions: swapping values and checking odd/even (3 hours)?
- How would you distinguish MO2.1 — Functions: swapping values and checking odd/even (3 hours) from a closely related idea?
- Where would an engineer or technician encounter MO2.1 — Functions: swapping values and checking odd/even (3 hours), and what must be checked before using it?
- What common mistake would make an answer or operation involving MO2.1 — Functions: swapping values and checking odd/even (3 hours) incorrect?

Common mistakes and safety emphasis

- Using a memorised relation without checking its conditions.
- Mixing prefixes or units, such as milli, kilo, mega, seconds, minutes, or degrees.
- Reporting a numerical value without a unit or without explaining what it means.
- Rounding intermediate values so aggressively that the final answer changes.

Malayalam support: MO2.1 — Functions: swapping values and checking odd/even (3 hours) □□□□ topic □□□□□□□□□□□□ □□□□□ exact technical term □□□□□□□□□□□□□. □□□□ definition □□□□□□□□□□ principle, named parts/steps, application, limitation □□□□□□ □□□□□□□□ □□□□□□□□. English terminology, symbols, units, diagram labels □□□□□□ □□□□□□□□□. Exam answer □□□□□□□□□□ concept-□□□□ □□□□□–□□□ □□□□□ □□□□□□□□□□□□□.

MO2.3 — Default arguments (3 hours)

Concept and explanation

A default argument supplies a value when the caller omits that parameter. It is written after required parameters, for example `def greet(name, message="Welcome")`. Default values should be immutable or deliberately handled; a mutable default list can retain data between calls and cause surprising behaviour.

Malayalam support: Argument □□□□□□□□□□□□ □□□□□□□□□□□□□ □□□□□□□□□ □□□□□ value □□□ default argument. Required argument-□□□□□□□ □□□□ default parameter □□□□□□.

Study points

- Call functions both with and without the optional value; place required parameters before defaults; avoid mutable defaults such as [] unless intentionally managed.

Engineering / workplace application: Configuration functions, report formatting, and optional search limits benefit from defaults.

Illustrative example: `def area(length, breadth=1): return length*breadth` calculates a rectangle area or a strip area when breadth is omitted.

Common mistake: Defining a required parameter after a default parameter raises a syntax error.

Handbook treatment

MO2.3 — Default arguments (3 hours) is an official examinable part of the module. Study it as a connected explanation: define the central term, identify the related elements, explain their relationship, and finish with a relevant engineering use or limitation.

Learning objectives

- Define and scope MO2.3 — Default arguments (3 hours) using the official terminology retained above.
- Organise the important elements of MO2.3 — Default arguments (3 hours) into a logical sequence, classification, structure, or calculation.
- Connect MO2.3 — Default arguments (3 hours) with one engineering decision, operation, measurement, or safety requirement in the context of Functions and Strings.
- Write an examination answer on MO2.3 — Default arguments (3 hours) with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading MO2.3 — Default arguments (3 hours). It is a study organisation method, not an additional official syllabus requirement.

- Start with the exact technical term and its scope.
- Separate the topic into named elements, stages, types, or conditions.
- Explain the relationship using cause-and-effect, sequence, comparison, or a labelled representation.
- Apply the idea to a realistic engineering situation and state one limitation or common mistake.

Engineering application lens

The engineering value of MO2.3 — Default arguments (3 hours) is understood by asking what requirement it satisfies, what input or condition it depends on, how the output is obtained, and how a technician or designer verifies the result. Use the module context to make the application specific rather than memorising an isolated definition.

Worked answer method

Given: the topic, operating condition, diagram, data, or situation stated in the question.
Required: definition, explanation, comparison, procedure, calculation, or application as requested.
Method: organise the answer under principle, named elements, sequence or relationship, and application.
Answer: use the requested technical terms, units, labels, assumptions, and a concluding statement.

Exam answer blueprint

For a short-answer question on MO2.3 — Default arguments (3 hours), begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is MO2.3 — Default arguments (3 hours), and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining MO2.3 — Default arguments (3 hours)?
- How would you distinguish MO2.3 — Default arguments (3 hours) from a closely related idea?
- Where would an engineer or technician encounter MO2.3 — Default arguments (3 hours), and what must be checked before using it?
- What common mistake would make an answer or operation involving MO2.3 — Default arguments (3 hours) incorrect?

Common mistakes and safety emphasis

- Copying a definition without explaining the mechanism or purpose.
- Listing terms without showing their relationship.
- Using an example that does not match the stated topic or condition.
- Leaving out a diagram, unit, assumption, limitation, or safety point when the question requires it.

Malayalam support: MO2.3 — Default arguments (3 hours) □□□□ topic □□□□□□□□□□□□ □□□□□ exact technical term □□□□□□□□□□□□□. □□□□ definition □□□□□□□□□□ principle, named parts/steps, application, limitation □□□□□□ □□□□□□□□ □□□□□□□□. English terminology, symbols, units, diagram labels □□□□□□ □□□□□□□□□. Exam answer □□□□□□□□□□ concept-□□□□ □□□□□–□□□ □□□□□ □□□□□□□□□□□□□.

MO2.3 — Strings (6 hours)

Concept and explanation

A string is an ordered, immutable sequence of characters. Indexing starts at zero, negative indices count from the end, and slicing uses start:stop:step with an exclusive stop. Common methods include lower, upper, strip, split, join, replace, find, and count. Concatenation and f-strings format text; input arrives as a string and may need conversion.

Malayalam support: String □□□□□□ characters-□□□□ immutable sequence □□□. Index 0 □□□□ □□□□□□□□; slicing-□ stop position □□□□□□□□□□□. Input-□□ numeric operation □□□□□□□ int/float □□□ convert □□□□□□□.

Study points

- Check empty strings; distinguish `find` returning -1 from an exception; use strip before comparing user input; prefer f-strings for readable output.

Engineering / workplace application: Names, commands, CSV fields, passwords, addresses, and report labels are all processed as strings.

Illustrative example: For `s="Python"`, `s[1:4]` is `yth`; `"-".join(["A","B"])` produces `A-B`.

Common mistake: Trying to change `s[0]` directly fails because strings are immutable; create a new string instead.

Handbook treatment

MO2.3 — Strings (6 hours) is an official examinable part of the module. Study it as a connected explanation: define the central term, identify the related elements, explain their relationship, and finish with a relevant engineering use or limitation.

Learning objectives

- Define and scope MO2.3 — Strings (6 hours) using the official terminology retained above.
- Organise the important elements of MO2.3 — Strings (6 hours) into a logical sequence, classification, structure, or calculation.
- Connect MO2.3 — Strings (6 hours) with one engineering decision, operation, measurement, or safety requirement in the context of Functions and Strings.
- Write an examination answer on MO2.3 — Strings (6 hours) with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading MO2.3 — Strings (6 hours). It is a study organisation method, not an additional official syllabus requirement.

- Start with the exact technical term and its scope.
- Separate the topic into named elements, stages, types, or conditions.
- Explain the relationship using cause-and-effect, sequence, comparison, or a labelled representation.
- Apply the idea to a realistic engineering situation and state one limitation or common mistake.

Engineering application lens

The engineering value of MO2.3 — Strings (6 hours) is understood by asking what requirement it satisfies, what input or condition it depends on, how the output is obtained, and how a technician or designer verifies the result. Use the module context to make the application specific rather than memorising an isolated definition.

Worked answer method

Given: the topic, operating condition, diagram, data, or situation stated in the question.
Required: definition, explanation, comparison, procedure, calculation, or application as requested.
Method: organise the answer under principle, named elements, sequence or relationship, and application.
Answer: use the requested technical terms, units, labels, assumptions, and a concluding statement.

Exam answer blueprint

For a short-answer question on MO2.3 — Strings (6 hours), begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is MO2.3 — Strings (6 hours), and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining MO2.3 — Strings (6 hours)?
- How would you distinguish MO2.3 — Strings (6 hours) from a closely related idea?
- Where would an engineer or technician encounter MO2.3 — Strings (6 hours), and what must be checked before using it?
- What common mistake would make an answer or operation involving MO2.3 — Strings (6 hours) incorrect?

Common mistakes and safety emphasis

- Copying a definition without explaining the mechanism or purpose.
- Listing terms without showing their relationship.
- Using an example that does not match the stated topic or condition.
- Leaving out a diagram, unit, assumption, limitation, or safety point when the question requires it.

Malayalam support: MO2.3 — Strings (6 hours) □□□□ topic □□□□□□□□□□□□ □□□□□ exact technical term □□□□□□□□□□□□□. □□□□ definition □□□□□□□□□□ principle, named parts/steps, application, limitation □□□□□□ □□□□□□□□ □□□□□□□□. English terminology, symbols, units, diagram labels □□□□□□ □□□□□□□□□. Exam answer □□□□□□□□□□ concept-□□□□ □□□□□–□□□ □□□□□ □□□□□□□□□□□□□□.

Module summary: Functions make solutions reusable; strings make text processing predictable and testable.

OFFICIAL MODULE 3

Lists, Dictionaries, Tuples and Sets

Python collections store groups of values. The correct data structure depends on whether order, mutability, duplicate values, or key-based access is most important.

Hours: 10
CO3 · Bloom level: Applying

Handbook study routine: Complete Module 3 in three passes. First read the official source coverage and topic headings. Next study every Handbook treatment, writing the key definition, relationship, procedure, formula, diagram, or comparison in your own words. Finally close the notes and answer the self-check questions and the module questions without looking at the answer.

Official source coverage for Module 3

Source basis: Official topic-row context. Every point below is retained and linked to a study-oriented explanation. The main topic cards remain the primary lesson narrative.

View extracted official source transcript

MO3.1 Implement Python programs using lists 3 Applying MO3.2 Implement Python programs using dictionaries 3 Applying
MO3.2 Implement Python programs using dictionaries 3 Applying MO3.3 Implement Python programs using lists 3 Applying
MO3.3 Implement Python programs using lists 3 Applying MO3.4 Implement Python programs using sets 1 Applying
MO3.4 Implement Python programs using sets 1 Applying 2

Point-by-point study guide

– Official syllabus point 1: MO3.1 Implement Python programs using lists 3 Applying MO3.2 Implement Python programs using dictionaries 3 Applying

Exact source point

> **Syllabus text:** MO3.1 Implement Python programs using lists 3 Applying MO3.2 Implement Python programs using dictionaries 3 Applying

How to understand and study it

This point is an official syllabus requirement: “MO3.1 Implement Python programs using lists 3 Applying MO3.2 Implement Python programs using dictionaries 3 Applying”. Study the input, logic or algorithm, output, and one test condition. Write the steps in order and identify the common error that changes the result. The main topic explanation above gives the concept context; this point-by-point section ensures that each named item is revised separately.

Study sequence

1. Write the exact technical term and a one-sentence definition in your own words.
2. Practise one small input-output example and test at least one boundary or fault condition.
3. Finish with one examination answer: definition, explanation, diagram/table if relevant, and application or limitation.

> **Malayalam support:** □ syllabus point □□□□□□□□□□□□ MO3.1 Implement Python programs using lists 3 Applying MO3.2 Implement Python programs using dictionaries 3 Applying □□□□ technical terms English-□□□□□□□ □□□□□□□. □□□□□ definition □□□□□□□□□□ principle □□□□□□□□□□□□□; □□□□ □□□ term-□□□□ function, difference, application □□□□□□ □□□□□□□□□□ □□□□□□. Diagram, sequence, formula, unit □□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□ revision □□□□□□□.

> **Exam focus:** Answer this point with its definition or principle first, followed by the named features, sequence, comparison, formula, diagram, or application that the question demands.

Handbook treatment

MO3.1 Implement Python programs using lists 3 Applying MO3.2 Implement Python programs using dictionaries 3 Applying should be understood as an information-flow problem. Identify the input, the rule or program that processes it, the internal state or decision, and the output. A correct explanation must show the sequence, not merely list software terms.

Learning objectives

- Define and scope MO3.1 Implement Python programs using lists 3 Applying MO3.2 Implement Python programs using dictionaries 3 Applying using the official terminology retained above.
- Organise the important elements of MO3.1 Implement Python programs using lists 3 Applying MO3.2 Implement Python programs using dictionaries 3 Applying into a logical sequence, classification, structure, or calculation.
- Connect MO3.1 Implement Python programs using lists 3 Applying MO3.2 Implement Python programs using dictionaries 3 Applying with one engineering decision, operation, measurement, or safety requirement in the context of Lists, Dictionaries, Tuples and Sets.
- Write an examination answer on MO3.1 Implement Python programs using lists 3 Applying MO3.2 Implement Python programs using dictionaries 3 Applying with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading MO3.1 Implement Python programs using lists 3 Applying MO3.2 Implement Python programs using dictionaries 3 Applying. It is a study organisation method, not an additional official syllabus requirement.

- List the input signals, data, or conditions.
- Write the logic, algorithm, instruction sequence, or protocol rule in the order it is evaluated.
- State the output and identify what changes when an input or state changes.
- Test a normal case, a boundary case, and a fault or invalid-input case.

Engineering application lens

For engineering practice, MO3.1 Implement Python programs using lists 3 Applying MO3.2 Implement Python programs using dictionaries 3 Applying matters because an automated or digital system must convert inputs into repeatable outputs. The technician should be able to trace a signal or data item, locate the decision that controls the result, and identify a safe response when an input is missing or abnormal.

Worked answer method

Given: the input conditions, required output, and any stated constraints.
Required: the logic sequence, program structure, truth table, or operating result.
Method: break the problem into named states or steps; evaluate them in order; test at least one alternate condition.
Answer: show the final sequence and state the expected output for each important condition.

Exam answer blueprint

For a short-answer question on MO3.1 Implement Python programs using lists 3 Applying MO3.2 Implement Python programs using dictionaries 3 Applying, begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is MO3.1 Implement Python programs using lists 3 Applying MO3.2 Implement Python programs using dictionaries 3 Applying, and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining MO3.1 Implement Python programs using lists 3 Applying MO3.2 Implement Python programs using dictionaries 3 Applying?
- How would you distinguish MO3.1 Implement Python programs using lists 3 Applying MO3.2 Implement Python programs using dictionaries 3 Applying from a closely related idea?
- Where would an engineer or technician encounter MO3.1 Implement Python programs using lists 3 Applying MO3.2 Implement Python programs using dictionaries 3 Applying, and what must be checked before using it?
- What common mistake would make an answer or operation involving MO3.1 Implement Python programs using lists 3 Applying MO3.2 Implement Python programs using dictionaries 3 Applying incorrect?

Common mistakes and safety emphasis

- Writing instructions without defining the input and output.
- Ignoring scan order, state retention, initialization, or boundary conditions.
- Confusing a logical condition with the physical signal that represents it.

- Testing only the normal case and failing to consider a fault or interlock condition.

> **Malayalam support:** MO3.1 Implement Python programs using lists 3 Applying MO3.2 Implement Python programs using dictionaries 3 Applying □□□□ topic □□□□□□□□□□□□ □□□□□ exact technical term □□□□□□□□□□□□□. □□□□ definition □□□□□□□□□□ principle, named parts/steps, application, limitation □□□□□□ □□□□□□□□ □□□□□□□□. English terminology, symbols, units, diagram labels □□□□□□ □□□□□□□□□. Exam answer □□□□□□□□□□ concept-□□□□ □□□□□– □□□ □□□□□ □□□□□□□□□□□□□.

– Official syllabus point 2: MO3.2 Implement Python programs using dictionaries 3 Applying MO3.3 Implement Python programs using lists 3 Applying

Exact source point

> **Syllabus text:** MO3.2 Implement Python programs using dictionaries 3 Applying MO3.3 Implement Python programs using lists 3 Applying

How to understand and study it

This point is an official syllabus requirement: “MO3.2 Implement Python programs using dictionaries 3 Applying MO3.3 Implement Python programs using lists 3 Applying”. Study the input, logic or algorithm, output, and one test condition. Write the steps in order and identify the common error that changes the result. The main topic explanation above gives the concept context; this point-by-point section ensures that each named item is revised separately.

Study sequence

1. Write the exact technical term and a one-sentence definition in your own words.
2. Practise one small input-output example and test at least one boundary or fault condition.
3. Finish with one examination answer: definition, explanation, diagram/table if relevant, and application or limitation.

> **Malayalam support:** □ syllabus point □□□□□□□□□□□□ MO3.2 Implement Python programs using dictionaries 3 Applying MO3.3 Implement Python programs using lists 3 Applying □□□□ technical terms English-□□□□□□□ □□□□□□□. □□□□□ definition □□□□□□□□□□ principle □□□□□□□□□□□□□; □□□□ □□□ term-□□□□ function, difference, application □□□□□□ □□□□□□□□□□ □□□□□□. Diagram, sequence, formula, unit □□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□ revision □□□□□□□.

> **Exam focus:** Answer this point with its definition or principle first, followed by the named features, sequence, comparison, formula, diagram, or application that the question demands.

Handbook treatment

MO3.2 Implement Python programs using dictionaries 3 Applying MO3.3 Implement Python programs using lists 3 Applying should be understood as an information-flow problem. Identify the input, the rule or program that processes it, the internal state or decision, and the output. A correct explanation must show the sequence, not merely list software terms.

Learning objectives

- Define and scope MO3.2 Implement Python programs using dictionaries 3 Applying MO3.3 Implement Python programs using lists 3 Applying using the official terminology retained above.
- Organise the important elements of MO3.2 Implement Python programs using dictionaries 3 Applying MO3.3 Implement Python programs using lists 3 Applying into a logical sequence, classification, structure, or calculation.
- Connect MO3.2 Implement Python programs using dictionaries 3 Applying MO3.3 Implement Python programs using lists 3 Applying with one engineering decision, operation, measurement, or safety requirement in the context of Lists, Dictionaries, Tuples and Sets.
- Write an examination answer on MO3.2 Implement Python programs using dictionaries 3 Applying MO3.3 Implement Python programs using lists 3 Applying with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading MO3.2 Implement Python programs using dictionaries 3 Applying MO3.3 Implement Python programs using lists 3 Applying. It is a study organisation method, not an additional official syllabus requirement.

- List the input signals, data, or conditions.
- Write the logic, algorithm, instruction sequence, or protocol rule in the order it is evaluated.
- State the output and identify what changes when an input or state changes.
- Test a normal case, a boundary case, and a fault or invalid-input case.

Engineering application lens

For engineering practice, MO3.2 Implement Python programs using dictionaries 3 Applying MO3.3 Implement Python programs using lists 3 Applying matters because an automated or digital system must convert inputs into repeatable outputs. The technician should be able to trace a signal or data item, locate the decision that controls the result, and identify a safe response when an input is missing or abnormal.

Worked answer method

Given: the input conditions, required output, and any stated constraints.
Required: the logic sequence, program structure, truth table, or operating result.
Method: break the problem into named states or steps; evaluate them in order; test at least one alternate condition.
Answer: show the final sequence and state the expected output for each important condition.

Exam answer blueprint

For a short-answer question on MO3.2 Implement Python programs using dictionaries 3 Applying MO3.3 Implement Python programs using lists 3 Applying, begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is MO3.2 Implement Python programs using dictionaries 3 Applying MO3.3 Implement Python programs using lists 3 Applying, and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining MO3.2 Implement Python programs using dictionaries 3 Applying MO3.3 Implement Python programs using lists 3 Applying?
- How would you distinguish MO3.2 Implement Python programs using dictionaries 3 Applying MO3.3 Implement Python programs using lists 3 Applying from a closely related idea?
- Where would an engineer or technician encounter MO3.2 Implement Python programs using dictionaries 3 Applying MO3.3 Implement Python programs using lists 3 Applying, and what must be checked before using it?
- What common mistake would make an answer or operation involving MO3.2 Implement Python programs using dictionaries 3 Applying MO3.3 Implement Python programs using lists 3 Applying incorrect?

Common mistakes and safety emphasis

- Writing instructions without defining the input and output.
- Ignoring scan order, state retention, initialization, or boundary conditions.
- Confusing a logical condition with the physical signal that represents it.

- Testing only the normal case and failing to consider a fault or interlock condition.

Malayalam support: MO3.2 Implement Python programs using dictionaries 3 Applying MO3.3 Implement Python programs using lists 3 Applying □□□□ topic □□□□□□□□□□□□ □□□□□ exact technical term □□□□□□□□□□□□□. □□□□ definition □□□□□□□□□□ principle, named parts/steps, application, limitation □□□□□□ □□□□□□□□ □□□□□□□□. English terminology, symbols, units, diagram labels □□□□□□ □□□□□□□□□. Exam answer □□□□□□□□□□ concept-□□□□ □□□□□–□□□ □□□□□ □□□□□□□□□□□□□□.

– Official syllabus point 3: MO3.3 Implement Python programs using lists 3 Applying MO3.4 Implement Python programs using sets 1 Applying

Exact source point

> **Syllabus text:** MO3.3 Implement Python programs using lists 3 Applying MO3.4 Implement Python programs using sets 1 Applying

How to understand and study it

This point is an official syllabus requirement: “MO3.3 Implement Python programs using lists 3 Applying MO3.4 Implement Python programs using sets 1 Applying”. Study the input, logic or algorithm, output, and one test condition. Write the steps in order and identify the common error that changes the result. The main topic explanation above gives the concept context; this point-by-point section ensures that each named item is revised separately.

Study sequence

1. Write the exact technical term and a one-sentence definition in your own words.
2. Practise one small input-output example and test at least one boundary or fault condition.
3. Finish with one examination answer: definition, explanation, diagram/table if relevant, and application or limitation.

> **Malayalam support:** □ syllabus point □□□□□□□□□□□□ MO3.3 Implement Python programs using lists 3 Applying MO3.4 Implement Python programs using sets 1 Applying □□□□ technical terms English-□□□□□□□ □□□□□□□. □□□□□ definition □□□□□□□□□□ principle □□□□□□□□□□□□□; □□□□ □□□ term-□□□□ function, difference, application □□□□□□ □□□□□□□□□□ □□□□□□. Diagram, sequence, formula, unit □□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□ revision □□□□□□□.

> **Exam focus:** Answer this point with its definition or principle first, followed by the named features, sequence, comparison, formula, diagram, or application that the question demands.

Handbook treatment

MO3.3 Implement Python programs using lists 3 Applying MO3.4 Implement Python programs using sets 1 Applying should be understood as an information-flow problem. Identify the input, the rule or program that processes it, the internal state or decision, and the output. A correct explanation must show the sequence, not merely list software terms.

Learning objectives

- Define and scope MO3.3 Implement Python programs using lists 3 Applying MO3.4 Implement Python programs using sets 1 Applying using the official terminology retained above.
- Organise the important elements of MO3.3 Implement Python programs using lists 3 Applying MO3.4 Implement Python programs using sets 1 Applying into a logical sequence, classification, structure, or calculation.
- Connect MO3.3 Implement Python programs using lists 3 Applying MO3.4 Implement Python programs using sets 1 Applying with one engineering decision, operation, measurement, or safety requirement in the context of Lists, Dictionaries, Tuples and Sets.
- Write an examination answer on MO3.3 Implement Python programs using lists 3 Applying MO3.4 Implement Python programs using sets 1 Applying with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading MO3.3 Implement Python programs using lists 3 Applying MO3.4 Implement Python programs using sets 1 Applying. It is a study organisation method, not an additional official syllabus requirement.

- List the input signals, data, or conditions.
- Write the logic, algorithm, instruction sequence, or protocol rule in the order it is evaluated.
- State the output and identify what changes when an input or state changes.
- Test a normal case, a boundary case, and a fault or invalid-input case.

Engineering application lens

For engineering practice, MO3.3 Implement Python programs using lists 3 Applying MO3.4 Implement Python programs using sets 1 Applying matters because an automated or digital system must convert inputs into repeatable outputs. The technician should be able to trace a signal or data item, locate the decision that controls the result, and identify a safe response when an input is missing or abnormal.

Worked answer method

Given: the input conditions, required output, and any stated constraints.
Required: the logic sequence, program structure, truth table, or operating result.
Method: break the problem into named states or steps; evaluate them in order; test at least one alternate condition.
Answer: show the final sequence and state the expected output for each important condition.

Exam answer blueprint

For a short-answer question on MO3.3 Implement Python programs using lists 3 Applying MO3.4 Implement Python programs using sets 1 Applying, begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is MO3.3 Implement Python programs using lists 3 Applying MO3.4 Implement Python programs using sets 1 Applying, and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining MO3.3 Implement Python programs using lists 3 Applying MO3.4 Implement Python programs using sets 1 Applying?
- How would you distinguish MO3.3 Implement Python programs using lists 3 Applying MO3.4 Implement Python programs using sets 1 Applying from a closely related idea?
- Where would an engineer or technician encounter MO3.3 Implement Python programs using lists 3 Applying MO3.4 Implement Python programs using sets 1 Applying, and what must be checked before using it?
- What common mistake would make an answer or operation involving MO3.3 Implement Python programs using lists 3 Applying MO3.4 Implement Python programs using sets 1 Applying incorrect?

Common mistakes and safety emphasis

- Writing instructions without defining the input and output.
- Ignoring scan order, state retention, initialization, or boundary conditions.
- Confusing a logical condition with the physical signal that represents it.
- Testing only the normal case and failing to consider a fault or interlock condition.

Malayalam support: MO3.3 Implement Python programs using lists 3 Applying MO3.4 Implement Python programs using sets 1 Applying □□□□ topic □□□□□□□□□□□□ □□□□□ exact technical term □□□□□□□□□□□□□. □□□□ definition □□□□□□□□□□ principle, named parts/steps, application, limitation □□□□□□ □□□□□□□□ □□□□□□□□. English terminology, symbols, units, diagram labels □□□□□□ □□□□□□□□□. Exam answer □□□□□□□□□□ concept-□□□□ □□□□□– □□□ □□□□□ □□□□□□□□□□□□□.

– Official syllabus point 4: MO3.4 Implement Python programs using sets 1 Applying 2

Exact source point

> **Syllabus text:** MO3.4 Implement Python programs using sets 1 Applying 2

How to understand and study it

This point is an official syllabus requirement: “MO3.4 Implement Python programs using sets 1 Applying 2”. Study the input, logic or algorithm, output, and one test condition. Write the steps in order and identify the common error that changes the result. The main topic explanation above gives the concept context; this point-by-point section ensures that each named item is revised separately.

Study sequence

1. Write the exact technical term and a one-sentence definition in your own words.
2. Practise one small input-output example and test at least one boundary or fault condition.
3. Finish with one examination answer: definition, explanation, diagram/table if relevant, and application or limitation.

> **Malayalam support:** □ syllabus point □□□□□□□□□□□□ MO3.4 Implement Python programs using sets 1 Applying 2 □□□□ technical terms English-□□□□□□□ □□□□□□□. □□□□□ definition □□□□□□□□□□ principle □□□□□□□□□□□□□; □□□□ □□□ term-□□□□ function, difference, application □□□□□□ □□□□□□□□□□ □□□□□□. Diagram, sequence, formula, unit □□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□ revision □□□□□□□.

> **Exam focus:** Answer this point with its definition or principle first, followed by the named features, sequence, comparison, formula, diagram, or application that the question demands.

Handbook treatment

MO3.4 Implement Python programs using sets 1 Applying 2 should be understood as an information-flow problem. Identify the input, the rule or program that processes it, the internal state or decision, and the output. A correct explanation must show the sequence, not merely list software terms.

Learning objectives

- Define and scope MO3.4 Implement Python programs using sets 1 Applying 2 using the official terminology retained above.
- Organise the important elements of MO3.4 Implement Python programs using sets 1 Applying 2 into a logical sequence, classification, structure, or calculation.
- Connect MO3.4 Implement Python programs using sets 1 Applying 2 with one engineering decision, operation, measurement, or safety requirement in the context of Lists, Dictionaries, Tuples and Sets.
- Write an examination answer on MO3.4 Implement Python programs using sets 1 Applying 2 with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading MO3.4 Implement Python programs using sets 1 Applying 2. It is a study organisation method, not an additional official syllabus requirement.

- List the input signals, data, or conditions.
- Write the logic, algorithm, instruction sequence, or protocol rule in the order it is evaluated.
- State the output and identify what changes when an input or state changes.
- Test a normal case, a boundary case, and a fault or invalid-input case.

Engineering application lens

For engineering practice, MO3.4 Implement Python programs using sets 1 Applying 2 matters because an automated or digital system must convert inputs into repeatable outputs. The technician should be able to trace a signal or data item, locate the decision that controls the result, and identify a safe response when an input is missing or abnormal.

Worked answer method

Given: the input conditions, required output, and any stated constraints.
Required: the logic sequence, program structure, truth table, or operating result.
Method: break the problem into named states or steps; evaluate them in order; test at least one alternate condition.
Answer: show the final sequence and state the expected output for each important condition.

Exam answer blueprint

For a short-answer question on MO3.4 Implement Python programs using sets 1 Applying 2, begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is MO3.4 Implement Python programs using sets 1 Applying 2, and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining MO3.4 Implement Python programs using sets 1 Applying 2?
- How would you distinguish MO3.4 Implement Python programs using sets 1 Applying 2 from a closely related idea?
- Where would an engineer or technician encounter MO3.4 Implement Python programs using sets 1 Applying 2, and what must be checked before using it?
- What common mistake would make an answer or operation involving MO3.4 Implement Python programs using sets 1 Applying 2 incorrect?

Common mistakes and safety emphasis

- Writing instructions without defining the input and output.
- Ignoring scan order, state retention, initialization, or boundary conditions.
- Confusing a logical condition with the physical signal that represents it.
- Testing only the normal case and failing to consider a fault or interlock condition.

Malayalam support: MO3.4 Implement Python programs using sets 1 Applying 2 □□□□ topic □□□□□□□□□□□□ □□□□□ exact technical term □□□□□□□□□□□□□. □□□□ definition □□□□□□□□□□ principle, named parts/steps, application, limitation □□□□□□ □□□□□□□□ □□□□□□□□. English terminology, symbols, units, diagram labels □□□□□□ □□□□□□□□□. Exam answer □□□□□□□□□□ concept-□□□□ □□□□□–□□□ □□□□□ □□□□□□□□□□□□□□.

Learning goals

- Create, index, slice, update, and traverse lists.
- Use dictionaries for key-value records and tuples for fixed grouped values.
- Use sets for unique items and set operations.

Module study tip

Read every topic in sequence, reproduce its example or procedure, and write a short explanation before attempting the module questions.

Learning map for Module module-3: the listed topics build the module competency.

MO3.1 — Lists (3 hours)

Concept and explanation

A list is an ordered, mutable collection written with square brackets. It may contain mixed data types, supports indexing and slicing, and provides append, extend, insert, remove, pop, sort, and reverse operations. Iterating over a list is clearer than repeatedly writing separate variables. Copying with `b=a` aliases the same list; `a.copy()` creates a shallow copy.

Malayalam support: List □□□□□□ order □□□□□□□□□□□□□, □□□□□□□ □□□□□□□□ collection □□□. append □□□ item □□□□□□□□; extend □□ items □□□□□□□□.

Study points

- Select list when order and change are required; check index range; use meaningful iteration variables; do not modify a list unpredictably while iterating.

Engineering / workplace application: Marks, inventory items, sensor readings, and menu items are naturally represented as lists.

Illustrative example: A list of marks can be averaged with `sum(marks)/len(marks)` after checking that the list is not empty.

Common mistake: `b=a` independent copy □□□□; b-□□□ □□□□□□ □□□□□□□□□□ a-□□□ □□□□□.

Handbook treatment

MO3.1 — Lists (3 hours) is an official examinable part of the module. Study it as a connected explanation: define the central term, identify the related elements, explain their relationship, and finish with a relevant engineering use or limitation.

Learning objectives

- Define and scope MO3.1 — Lists (3 hours) using the official terminology retained above.
- Organise the important elements of MO3.1 — Lists (3 hours) into a logical sequence, classification, structure, or calculation.
- Connect MO3.1 — Lists (3 hours) with one engineering decision, operation, measurement, or safety requirement in the context of Lists, Dictionaries, Tuples and Sets.
- Write an examination answer on MO3.1 — Lists (3 hours) with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading MO3.1 — Lists (3 hours). It is a study organisation method, not an additional official syllabus requirement.

- Start with the exact technical term and its scope.
- Separate the topic into named elements, stages, types, or conditions.
- Explain the relationship using cause-and-effect, sequence, comparison, or a labelled representation.
- Apply the idea to a realistic engineering situation and state one limitation or common mistake.

Engineering application lens

The engineering value of MO3.1 — Lists (3 hours) is understood by asking what requirement it satisfies, what input or condition it depends on, how the output is obtained, and how a technician or designer verifies the result. Use the module context to make the application specific rather than memorising an isolated definition.

Worked answer method

Given: the topic, operating condition, diagram, data, or situation stated in the question.
Required: definition, explanation, comparison, procedure, calculation, or application as requested.
Method: organise the answer under principle, named elements, sequence or relationship, and application.
Answer: use the requested technical terms, units, labels, assumptions, and a concluding statement.

Exam answer blueprint

For a short-answer question on MO3.1 — Lists (3 hours), begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is MO3.1 — Lists (3 hours), and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining MO3.1 — Lists (3 hours)?
- How would you distinguish MO3.1 — Lists (3 hours) from a closely related idea?
- Where would an engineer or technician encounter MO3.1 — Lists (3 hours), and what must be checked before using it?
- What common mistake would make an answer or operation involving MO3.1 — Lists (3 hours) incorrect?

Common mistakes and safety emphasis

- Copying a definition without explaining the mechanism or purpose.
- Listing terms without showing their relationship.
- Using an example that does not match the stated topic or condition.
- Leaving out a diagram, unit, assumption, limitation, or safety point when the question requires it.

Malayalam support: MO3.1 — Lists (3 hours) □□□□ topic □□□□□□□□□□□□ □□□□□ exact technical term □□□□□□□□□□□□□. □□□□ definition □□□□□□□□□□ principle, named parts/steps, application, limitation □□□□□□ □□□□□□□□ □□□□□□□□. English terminology, symbols, units, diagram labels □□□□□□ □□□□□□□□□. Exam answer □□□□□□□□□□ concept-□□□□ □□□□□–□□□ □□□□□ □□□□□□□□□□□□□□.

MO3.2 — Dictionaries (3 hours)

Concept and explanation

A dictionary stores key-value pairs. Keys must be hashable and unique; values may be any suitable object. Access can use `d[key]` when the key must exist or `d.get(key, default)` when absence is expected. Items, keys, and values views support iteration. Dictionaries are useful for records and fast lookup.

Malayalam support: Dictionary-□ key □□□□□□□□□□ value □□□□□□□□□□. Key unique □□□□□□□□□□. Key □□□□□□□□□□□□□□ `get()` □□□□□□□□□□□□□□□□ safer □□□.

Study points

- Use stable, descriptive keys; validate required keys; avoid duplicate keys in literals; iterate with items when both key and value are needed.

Engineering / workplace application: Student records, product prices, configuration settings, and frequency counts use dictionaries.

Illustrative example: `price={"pen":10,"book":50}` gives 50 for `price["book"]`; `price.get("bag",0)` safely returns zero if bag is absent.

Common mistake: Accessing a missing key with brackets raises KeyError; `get` with a default can handle optional data.

Handbook treatment

MO3.2 — Dictionaries (3 hours) is an official examinable part of the module. Study it as a connected explanation: define the central term, identify the related elements, explain their relationship, and finish with a relevant engineering use or limitation.

Learning objectives

- Define and scope MO3.2 — Dictionaries (3 hours) using the official terminology retained above.
- Organise the important elements of MO3.2 — Dictionaries (3 hours) into a logical sequence, classification, structure, or calculation.
- Connect MO3.2 — Dictionaries (3 hours) with one engineering decision, operation, measurement, or safety requirement in the context of Lists, Dictionaries, Tuples and Sets.
- Write an examination answer on MO3.2 — Dictionaries (3 hours) with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading MO3.2 — Dictionaries (3 hours). It is a study organisation method, not an additional official syllabus requirement.

- Start with the exact technical term and its scope.
- Separate the topic into named elements, stages, types, or conditions.
- Explain the relationship using cause-and-effect, sequence, comparison, or a labelled representation.
- Apply the idea to a realistic engineering situation and state one limitation or common mistake.

Engineering application lens

The engineering value of MO3.2 — Dictionaries (3 hours) is understood by asking what requirement it satisfies, what input or condition it depends on, how the output is obtained, and how a technician or designer verifies the result. Use the module context to make the application specific rather than memorising an isolated definition.

Worked answer method

Given: the topic, operating condition, diagram, data, or situation stated in the question.
Required: definition, explanation, comparison, procedure, calculation, or application as requested.
Method: organise the answer under principle, named elements, sequence or relationship, and application.
Answer: use the requested technical terms, units, labels, assumptions, and a concluding statement.

Exam answer blueprint

For a short-answer question on MO3.2 — Dictionaries (3 hours), begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is MO3.2 — Dictionaries (3 hours), and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining MO3.2 — Dictionaries (3 hours)?
- How would you distinguish MO3.2 — Dictionaries (3 hours) from a closely related idea?
- Where would an engineer or technician encounter MO3.2 — Dictionaries (3 hours), and what must be checked before using it?
- What common mistake would make an answer or operation involving MO3.2 — Dictionaries (3 hours) incorrect?

Common mistakes and safety emphasis

- Copying a definition without explaining the mechanism or purpose.
- Listing terms without showing their relationship.
- Using an example that does not match the stated topic or condition.
- Leaving out a diagram, unit, assumption, limitation, or safety point when the question requires it.

Malayalam support: MO3.2 — Dictionaries (3 hours) □□□□ topic □□□□□□□□□□□□□ □□□□□ exact technical term □□□□□□□□□□□□□. □□□□ definition □□□□□□□□□□ principle, named parts/steps, application, limitation □□□□□□ □□□□□□□□ □□□□□□□□. English terminology, symbols, units, diagram labels □□□□□□ □□□□□□□□□. Exam answer □□□□□□□□□□ concept-□□□□ □□□□□–□□□ □□□□□ □□□□□□□□□□□□□□.

MO3.3 — Lists: nested and practical operations (3 hours)

Concept and explanation

The supplied outline repeats the word lists for MO3.3. This lesson preserves that official label and uses the topic for nested lists, list comprehensions, filtering, and practical traversal. A nested list is a list whose elements are themselves lists; access requires one index per level.

Malayalam support: □ syllabus-□ MO3.3-□ lists □□□□□□□ □□□□□□□□□□□□□□□. □□□□□ nested lists, filtering, practical traversal □□□□ □□□□□□□ □□□□□□□□.

Study points

- Represent tables as nested lists; verify row lengths; use comprehensions only when they remain readable; keep transformations separate from input.

Engineering / workplace application: A marks table, pixel grid, timetable, or matrix can be modelled as a nested list.

Illustrative example: For `rows=[[1,2],[3,4]]`, `rows[1][0]` is 3; `[x*x for x in range(4)]` creates square values.

Common mistake: Assuming every row has the same length without checking causes IndexError on irregular data.

Handbook treatment

MO3.3 — Lists: nested and practical operations (3 hours) is an official examinable part of the module. Study it as a connected explanation: define the central term, identify the related elements, explain their relationship, and finish with a relevant engineering use or limitation.

Learning objectives

- Define and scope MO3.3 — Lists: nested and practical operations (3 hours) using the official terminology retained above.
- Organise the important elements of MO3.3 — Lists: nested and practical operations (3 hours) into a logical sequence, classification, structure, or calculation.
- Connect MO3.3 — Lists: nested and practical operations (3 hours) with one engineering decision, operation, measurement, or safety requirement in the context of Lists, Dictionaries, Tuples and Sets.
- Write an examination answer on MO3.3 — Lists: nested and practical operations (3 hours) with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading MO3.3 — Lists: nested and practical operations (3 hours). It is a study organisation method, not an additional official syllabus requirement.

- Start with the exact technical term and its scope.
- Separate the topic into named elements, stages, types, or conditions.
- Explain the relationship using cause-and-effect, sequence, comparison, or a labelled representation.
- Apply the idea to a realistic engineering situation and state one limitation or common mistake.

Engineering application lens

The engineering value of MO3.3 — Lists: nested and practical operations (3 hours) is understood by asking what requirement it satisfies, what input or condition it depends on, how the output is obtained, and how a technician or designer verifies the result. Use the module context to make the application specific rather than memorising an isolated definition.

Worked answer method

Given: the topic, operating condition, diagram, data, or situation stated in the question.
Required: definition, explanation, comparison, procedure, calculation, or application as requested.
Method: organise the answer under principle, named elements, sequence or relationship, and application.
Answer: use the requested technical terms, units, labels, assumptions, and a concluding statement.

Exam answer blueprint

For a short-answer question on MO3.3 — Lists: nested and practical operations (3 hours), begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is MO3.3 — Lists: nested and practical operations (3 hours), and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining MO3.3 — Lists: nested and practical operations (3 hours)?
- How would you distinguish MO3.3 — Lists: nested and practical operations (3 hours) from a closely related idea?
- Where would an engineer or technician encounter MO3.3 — Lists: nested and practical operations (3 hours), and what must be checked before using it?
- What common mistake would make an answer or operation involving MO3.3 — Lists: nested and practical operations (3 hours) incorrect?

Common mistakes and safety emphasis

- Copying a definition without explaining the mechanism or purpose.
- Listing terms without showing their relationship.
- Using an example that does not match the stated topic or condition.
- Leaving out a diagram, unit, assumption, limitation, or safety point when the question requires it.

Malayalam support: MO3.3 — Lists: nested and practical operations (3 hours) □□□□ topic □□□□□□□□□□□□ □□□□□ exact technical term □□□□□□□□□□□□□. □□□□ definition □□□□□□□□□□ principle, named parts/steps, application, limitation □□□□□□ □□□□□□□□ □□□□□□□□. English terminology, symbols, units, diagram labels □□□□□□ □□□□□□□□□. Exam answer □□□□□□□□□□ concept-□□□□ □□□□□–□□□ □□□□□ □□□□□□□□□□□□□.

MO3.4 — Sets (1 hours)

Concept and explanation

A set is an unordered collection of unique hashable elements. It supports add, remove/discard, union, intersection, difference, and membership testing. Sets are ideal for eliminating duplicates, but they do not preserve a meaningful display order.

Malayalam support: Set-□ duplicate values □□□□□□□□□□□□□□; order-□□ □□□□□□□□□□□□□□. Union, intersection □□□□□□□□ operations □□□□□□□□□□ groups compare □□□□□□□□.

Study points

- Use a set for uniqueness or membership; use discard when missing items are acceptable; convert to a sorted list when ordered output is needed.

Engineering / workplace application: Removing duplicate registration numbers or finding common skills between teams are set applications.

Illustrative example: `set([1,1,2,3])` becomes `{1,2,3}`; `{1,2}&{2,3}` gives `{2}`.

Common mistake: Indexing a set such as s[0] is invalid because sets are not sequences.

Handbook treatment

MO3.4 — Sets (1 hours) is an official examinable part of the module. Study it as a connected explanation: define the central term, identify the related elements, explain their relationship, and finish with a relevant engineering use or limitation.

Learning objectives

- Define and scope MO3.4 — Sets (1 hours) using the official terminology retained above.
- Organise the important elements of MO3.4 — Sets (1 hours) into a logical sequence, classification, structure, or calculation.
- Connect MO3.4 — Sets (1 hours) with one engineering decision, operation, measurement, or safety requirement in the context of Lists, Dictionaries, Tuples and Sets.
- Write an examination answer on MO3.4 — Sets (1 hours) with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading MO3.4 — Sets (1 hours). It is a study organisation method, not an additional official syllabus requirement.

- Start with the exact technical term and its scope.
- Separate the topic into named elements, stages, types, or conditions.
- Explain the relationship using cause-and-effect, sequence, comparison, or a labelled representation.
- Apply the idea to a realistic engineering situation and state one limitation or common mistake.

Engineering application lens

The engineering value of MO3.4 — Sets (1 hours) is understood by asking what requirement it satisfies, what input or condition it depends on, how the output is obtained, and how a technician or designer verifies the result. Use the module context to make the application specific rather than memorising an isolated definition.

Worked answer method

Given: the topic, operating condition, diagram, data, or situation stated in the question.
Required: definition, explanation, comparison, procedure, calculation, or application as requested.
Method: organise the answer under principle, named elements, sequence or relationship, and application.
Answer: use the requested technical terms, units, labels, assumptions, and a concluding statement.

Exam answer blueprint

For a short-answer question on MO3.4 — Sets (1 hours), begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is MO3.4 — Sets (1 hours), and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining MO3.4 — Sets (1 hours)?
- How would you distinguish MO3.4 — Sets (1 hours) from a closely related idea?
- Where would an engineer or technician encounter MO3.4 — Sets (1 hours), and what must be checked before using it?
- What common mistake would make an answer or operation involving MO3.4 — Sets (1 hours) incorrect?

Common mistakes and safety emphasis

- Copying a definition without explaining the mechanism or purpose.
- Listing terms without showing their relationship.
- Using an example that does not match the stated topic or condition.
- Leaving out a diagram, unit, assumption, limitation, or safety point when the question requires it.

Malayalam support: MO3.4 — Sets (1 hours) □□□□ topic □□□□□□□□□□□□□ □□□□□□ exact technical term □□□□□□□□□□□□□□. □□□□ definition □□□□□□□□□□□ principle, named parts/steps, application, limitation □□□□□□ □□□□□□□□ □□□□□□□□□. English terminology, symbols, units, diagram labels □□□□□□ □□□□□□□□□□. Exam answer □□□□□□□□□□ concept-□□□□ □□□□□–□□□ □□□□□ □□□□□□□□□□□□□□□.

Module summary: Choose collections by required order, mutability, uniqueness, and access method.

OFFICIAL MODULE 4

Files and Exception Handling

Programs become useful when they can preserve information and recover from expected errors. Files provide persistent storage; exceptions let a program handle failures without hiding genuine defects.

Hours: 10
CO4 · Bloom level: Applying

Handbook study routine: Complete Module 4 in three passes. First read the official source coverage and topic headings. Next study every Handbook treatment, writing the key definition, relationship, procedure, formula, diagram, or comparison in your own words. Finally close the notes and answer the self-check questions and the module questions without looking at the answer.

Official source coverage for Module 4

Source basis: Official topic-row context. Every point below is retained and linked to a study-oriented explanation. The main topic cards remain the primary lesson narrative.

View extracted official source transcript

MO4.1 Develop programs to illustrate exception handling 4 Applying Implement programs to create, and to open and MO4.2 3 Applying close a file
MO4.2 3 Applying close a file Develop programs to append a file and similar MO4.3 3 Applying
MO4.3 3 Applying other functions. Lab Exam – II 1½

Point-by-point study guide

– Official syllabus point 1: MO4.1 Develop programs to illustrate exception handling 4 Applying Implement programs to create, and to open and MO4.2 3 Applying close a file

Exact source point

> **Syllabus text:** MO4.1 Develop programs to illustrate exception handling 4 Applying Implement programs to create, and to open and MO4.2 3 Applying close a file

How to understand and study it

This point is an official syllabus requirement: “MO4.1 Develop programs to illustrate exception handling 4 Applying Implement programs to create, and to open and MO4.2 3 Applying close a file”. Study the input, logic or algorithm, output, and one test condition. Write the steps in order and identify the common error that changes the result. The main topic explanation above gives the concept context; this point-by-point section ensures that each named item is revised separately.

Study sequence

1. Write the exact technical term and a one-sentence definition in your own words.
2. Practise one small input-output example and test at least one boundary or fault condition.
3. Finish with one examination answer: definition, explanation, diagram/table if relevant, and application or limitation.

> **Malayalam support:** □ syllabus point □□□□□□□□□□□□ MO4.1 Develop programs to illustrate exception handling 4 Applying Implement programs to create, and to open, MO4.2 3 Applying close a file □□□□ technical terms English-□□□□□□□□ □□□□□□□. □□□□□ definition □□□□□□□□□□ principle □□□□□□□□□□□□□; □□□□ □□□ term-□□□□ function, difference, application □□□□□□ □□□□□□□□□□ □□□□□□. Diagram, sequence, formula, unit □□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□ revision □□□□□□□.

> **Exam focus:** Answer this point with its definition or principle first, followed by the named features, sequence, comparison, formula, diagram, or application that the question demands.

Handbook treatment

MO4.1 Develop programs to illustrate exception handling 4 Applying Implement programs to create, and to open and MO4.2 3 Applying close a file should be understood as an information-flow problem. Identify the input, the rule or program that processes it, the internal state or decision, and the output. A correct explanation must show the sequence, not merely list software terms.

Learning objectives

- Define and scope MO4.1 Develop programs to illustrate exception handling 4 Applying Implement programs to create, and to open and MO4.2 3 Applying close a file using the official terminology retained above.
- Organise the important elements of MO4.1 Develop programs to illustrate exception handling 4 Applying Implement programs to create, and to open and MO4.2 3 Applying close a file into a logical sequence, classification, structure, or calculation.
- Connect MO4.1 Develop programs to illustrate exception handling 4 Applying Implement programs to create, and to open and MO4.2 3 Applying close a file with one engineering decision, operation, measurement, or safety requirement in the context of Files and Exception Handling.
- Write an examination answer on MO4.1 Develop programs to illustrate exception handling 4 Applying Implement programs to create, and to open and MO4.2 3 Applying close a file with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading MO4.1 Develop programs to illustrate exception handling 4 Applying Implement programs to create, and to open and MO4.2 3 Applying close a file. It is a study organisation method, not an additional official syllabus requirement.

- List the input signals, data, or conditions.
- Write the logic, algorithm, instruction sequence, or protocol rule in the order it is evaluated.
- State the output and identify what changes when an input or state changes.
- Test a normal case, a boundary case, and a fault or invalid-input case.

Engineering application lens

For engineering practice, MO4.1 Develop programs to illustrate exception handling 4 Applying Implement programs to create, and to open and MO4.2 3 Applying close a file matters because an automated or digital system must convert inputs into repeatable outputs. The technician should be able to trace a

signal or data item, locate the decision that controls the result, and identify a safe response when an input is missing or abnormal.

Worked answer method

Given: the input conditions, required output, and any stated constraints.
Required: the logic sequence, program structure, truth table, or operating result.
Method: break the problem into named states or steps; evaluate them in order; test at least one alternate condition.
Answer: show the final sequence and state the expected output for each important condition.

Exam answer blueprint

For a short-answer question on MO4.1 Develop programs to illustrate exception handling 4 Applying Implement programs to create, and to open and MO4.2 3 Applying close a file, begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is MO4.1 Develop programs to illustrate exception handling 4 Applying Implement programs to create, and to open and MO4.2 3 Applying close a file, and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining MO4.1 Develop programs to illustrate exception handling 4 Applying Implement programs to create, and to open and MO4.2 3 Applying close a file?
- How would you distinguish MO4.1 Develop programs to illustrate exception handling 4 Applying Implement programs to create, and to open and MO4.2 3 Applying close a file from a closely related idea?
- Where would an engineer or technician encounter MO4.1 Develop programs to illustrate exception handling 4 Applying Implement programs to create, and to open and MO4.2 3 Applying close a file, and what must be checked before using it?
- What common mistake would make an answer or operation involving MO4.1 Develop programs to illustrate exception handling 4 Applying Implement programs to create, and to open and MO4.2 3 Applying close a file incorrect?

Common mistakes and safety emphasis

- Writing instructions without defining the input and output.
- Ignoring scan order, state retention, initialization, or boundary conditions.
- Confusing a logical condition with the physical signal that represents it.
- Testing only the normal case and failing to consider a fault or interlock condition.

Malayalam support: MO4.1 Develop programs to illustrate exception handling 4 Applying Implement programs to create, and to open and MO4.2 3 Applying close a file □□□□ topic □□□□□□□□□□□□ □□□□□ exact technical term □□□□□□□□□□□□□. □□□□ definition □□□□□□□□□□ principle, named parts/steps, application, limitation □□□□□□ □□□□□□□□ □□□□□□□□. English terminology, symbols, units, diagram labels □□□□□□ □□□□□□□□□. Exam answer □□□□□□□□□□ concept-□□□□ □□□□□–□□□ □□□□□ □□□□□□□□□□□□□□.

– Official syllabus point 2: MO4.2 3 Applying close a file Develop programs to append a file and similar MO4.3 3 Applying

Exact source point

> **Syllabus text:** MO4.2 3 Applying close a file Develop programs to append a file and similar MO4.3 3 Applying

How to understand and study it

This point is an official syllabus requirement: “MO4.2 3 Applying close a file Develop programs to append a file and similar MO4.3 3 Applying”. Study the input, logic or algorithm, output, and one test condition. Write the steps in order and identify the common error that changes the result. The main topic explanation above gives the concept context; this point-by-point section ensures that each named item is revised separately.

Study sequence

1. Write the exact technical term and a one-sentence definition in your own words.
2. Practise one small input-output example and test at least one boundary or fault condition.
3. Finish with one examination answer: definition, explanation, diagram/table if relevant, and application or limitation.

> **Malayalam support:** □ syllabus point □□□□□□□□□□□□ MO4.2 3 Applying close a file Develop programs to append a file, similar MO4.3 3 Applying □□□□ technical terms English-□□□□□□□□ □□□□□□□□. □□□□□ definition □□□□□□□□□□ principle □□□□□□□□□□□□□□; □□□□ □□□ term-□□□□ function, difference, application □□□□□□ □□□□□□□□□□ □□□□□□. Diagram, sequence, formula, unit □□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□ revision □□□□□□□□.

> **Exam focus:** Answer this point with its definition or principle first, followed by the named features, sequence, comparison, formula, diagram, or application that the question demands.

Handbook treatment

MO4.2 3 Applying close a file Develop programs to append a file and similar MO4.3 3 Applying should be understood as an information-flow problem. Identify the input, the rule or program that processes it, the internal state or decision, and the output. A correct explanation must show the sequence, not merely list software terms.

Learning objectives

- Define and scope MO4.2 3 Applying close a file Develop programs to append a file and similar MO4.3 3 Applying using the official terminology retained above.
- Organise the important elements of MO4.2 3 Applying close a file Develop programs to append a file and similar MO4.3 3 Applying into a logical sequence, classification, structure, or calculation.
- Connect MO4.2 3 Applying close a file Develop programs to append a file and similar MO4.3 3 Applying with one engineering decision, operation, measurement, or safety requirement in the context of Files and Exception Handling.
- Write an examination answer on MO4.2 3 Applying close a file Develop programs to append a file and similar MO4.3 3 Applying with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading MO4.2 3 Applying close a file Develop programs to append a file and similar MO4.3 3 Applying. It is a study organisation method, not an additional official syllabus requirement.

- List the input signals, data, or conditions.
- Write the logic, algorithm, instruction sequence, or protocol rule in the order it is evaluated.
- State the output and identify what changes when an input or state changes.
- Test a normal case, a boundary case, and a fault or invalid-input case.

Engineering application lens

For engineering practice, MO4.2 3 Applying close a file Develop programs to append a file and similar MO4.3 3 Applying matters because an automated or digital system must convert inputs into repeatable outputs. The technician should be able to trace a signal or data item, locate the decision that controls the result, and identify a safe response when an input is missing or abnormal.

Worked answer method

Given: the input conditions, required output, and any stated constraints.
Required: the logic sequence, program structure, truth table, or operating result.
Method: break the problem into named states or steps; evaluate them in order; test at least one alternate condition.
Answer: show the final sequence and state the expected output for each important condition.

Exam answer blueprint

For a short-answer question on MO4.2 3 Applying close a file Develop programs to append a file and similar MO4.3 3 Applying, begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is MO4.2 3 Applying close a file Develop programs to append a file and similar MO4.3 3 Applying, and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining MO4.2 3 Applying close a file Develop programs to append a file and similar MO4.3 3 Applying?
- How would you distinguish MO4.2 3 Applying close a file Develop programs to append a file and similar MO4.3 3 Applying from a closely related idea?
- Where would an engineer or technician encounter MO4.2 3 Applying close a file Develop programs to append a file and similar MO4.3 3 Applying, and what must be checked before using it?
- What common mistake would make an answer or operation involving MO4.2 3 Applying close a file Develop programs to append a file and similar MO4.3 3 Applying incorrect?

Common mistakes and safety emphasis

- Writing instructions without defining the input and output.
- Ignoring scan order, state retention, initialization, or boundary conditions.
- Confusing a logical condition with the physical signal that represents it.
- Testing only the normal case and failing to consider a fault or interlock condition.

Malayalam support: MO4.2 3 Applying close a file Develop programs to append a file and similar MO4.3 3 Applying □□□□ topic □□□□□□□□□□□□ □□□□□ exact technical term □□□□□□□□□□□□□. □□□□ definition □□□□□□□□□□ principle, named parts/steps, application, limitation □□□□□□ □□□□□□□□ □□□□□□□□. English terminology, symbols, units, diagram labels □□□□□□ □□□□□□□□□. Exam answer □□□□□□□□□□ concept-□□□□ □□□□□–□□□ □□□□□ □□□□□□□□□□□□□□.

– Official syllabus point 3: MO4.3 3 Applying other functions. Lab Exam – II 1½

Exact source point

> **Syllabus text:** MO4.3 3 Applying other functions. Lab Exam – II 1½

How to understand and study it

This point is an official syllabus requirement: “MO4.3 3 Applying other functions. Lab Exam – II 1½”. Treat every named term as an examinable subpoint. Define it, explain its role or behaviour, distinguish it from related terms, and connect it to the module application. The main topic explanation above gives the concept context; this point-by-point section ensures that each named item is revised separately.

Study sequence

1. Write the exact technical term and a one-sentence definition in your own words.
2. List the named terms separately and add one distinguishing feature or engineering use for each.
3. Finish with one examination answer: definition, explanation, diagram/table if relevant, and application or limitation.

> **Malayalam support:** □ syllabus point □□□□□□□□□□□□ MO4.3 3 Applying other functions. Lab Exam – II 1½ □□□□ technical terms English-□□□□□□□ □□□□□□□. □□□□□ definition □□□□□□□□□□ principle □□□□□□□□□□□□□; □□□□ □□□ term-□□□□ function, difference, application □□□□□□ □□□□□□□□□□ □□□□□□. Diagram, sequence, formula, unit □□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□ revision □□□□□□□.

> **Exam focus:** Answer this point with its definition or principle first, followed by the named features, sequence, comparison, formula, diagram, or application that the question demands.

Handbook treatment

MO4.3 3 Applying other functions. Lab Exam – II 1½ is an official examinable part of the module. Study it as a connected explanation: define the central term, identify the related elements, explain their relationship, and finish with a relevant engineering use or limitation.

Learning objectives

- Define and scope MO4.3 3 Applying other functions. Lab Exam – II 1½ using the official terminology retained above.
- Organise the important elements of MO4.3 3 Applying other functions. Lab Exam – II 1½ into a logical sequence, classification, structure, or calculation.
- Connect MO4.3 3 Applying other functions. Lab Exam – II 1½ with one engineering decision, operation, measurement, or safety requirement in the context of Files and Exception Handling.
- Write an examination answer on MO4.3 3 Applying other functions. Lab Exam – II 1½ with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading MO4.3 3 Applying other functions. Lab Exam – II 1½. It is a study organisation method, not an additional official syllabus requirement.

- Start with the exact technical term and its scope.
- Separate the topic into named elements, stages, types, or conditions.
- Explain the relationship using cause-and-effect, sequence, comparison, or a labelled representation.
- Apply the idea to a realistic engineering situation and state one limitation or common mistake.

Engineering application lens

The engineering value of MO4.3 3 Applying other functions. Lab Exam – II 1½ is understood by asking what requirement it satisfies, what input or condition it depends on, how the output is obtained, and how a technician or designer verifies the result. Use the module context to make the application specific rather than memorising an isolated definition.

Worked answer method

Given: the topic, operating condition, diagram, data, or situation stated in the question.
Required: definition, explanation, comparison, procedure, calculation, or application as requested.
Method: organise the answer under principle, named elements, sequence or relationship, and application.
Answer: use the requested technical terms, units, labels, assumptions, and a concluding statement.

Exam answer blueprint

For a short-answer question on MO4.3 3 Applying other functions. Lab Exam – II 1½, begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is MO4.3 3 Applying other functions. Lab Exam – II 1½, and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining MO4.3 3 Applying other functions. Lab Exam – II 1½?
- How would you distinguish MO4.3 3 Applying other functions. Lab Exam – II 1½ from a closely related idea?
- Where would an engineer or technician encounter MO4.3 3 Applying other functions. Lab Exam – II 1½, and what must be checked before using it?
- What common mistake would make an answer or operation involving MO4.3 3 Applying other functions. Lab Exam – II 1½ incorrect?

Common mistakes and safety emphasis

- Copying a definition without explaining the mechanism or purpose.
- Listing terms without showing their relationship.
- Using an example that does not match the stated topic or condition.
- Leaving out a diagram, unit, assumption, limitation, or safety point when the question requires it.

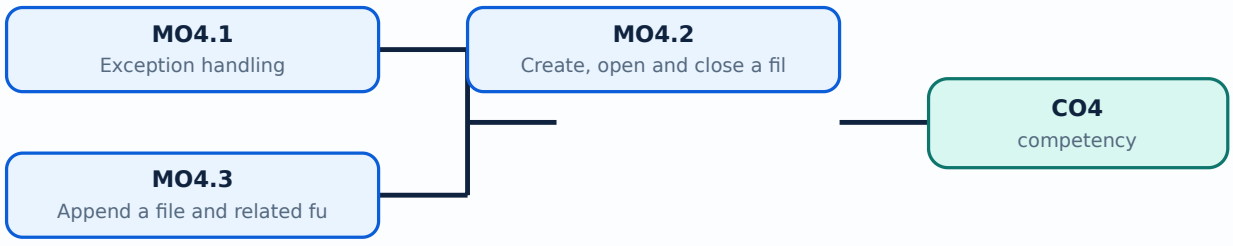
Malayalam support: MO4.3 3 Applying other functions. Lab Exam – II 1½ □□□□ topic □□□□□□□□□□□□ □□□□□ exact technical term □□□□□□□□□□□□□. □□□□ definition □□□□□□□□□□ principle, named parts/steps, application, limitation □□□□□□ □□□□□□□□ □□□□□□□□. English terminology, symbols, units, diagram labels □□□□□□ □□□□□□□□□. Exam answer □□□□□□□□□□ concept-□□□□ □□□□□– □□□ □□□□□ □□□□□□□□□□□□□.

Learning goals

- Use try, except, else, and finally appropriately.
- Create, open, read, write, append, and close text files.
- Use context managers and validate file operations.

Module study tip

Read every topic in sequence, reproduce its example or procedure, and write a short explanation before attempting the module questions.



Learning map for Module module-4: the listed topics build the module competency.

MO4.1 — Exception handling (4 hours)

Concept and explanation

An exception is an event that interrupts normal execution, such as ValueError, TypeError, IndexError, KeyError, or FileNotFoundError. A try block contains risky operations; except handles a known failure; else runs when no exception occurs; finally runs for cleanup. Catch specific exceptions rather than using a bare except, and do not use exception handling to mask incorrect logic.

Malayalam support: Exception □□□□□□□□□□□□□□□ program □□□□□□□□□□ □□□□□□□ expected error handle □□□□□□□ try/except □□□□□□□□□□□□□□□. Specific exception catch □□□□□□□□□□□□□ □□□□□□□.

Study points

- Validate before conversion; catch only expected exceptions; show a useful message; preserve cleanup in finally or a context manager; never expose sensitive data in error messages.

Engineering / workplace application: Input validation, file access, network requests, and database operations all need controlled error handling.

Illustrative example: For `int(input())`, catch ValueError and ask for a whole number again; do not catch every possible exception as ValueError.

Common mistake: A bare `except` can hide programming errors and make debugging difficult.

Handbook treatment

MO4.1 — Exception handling (4 hours) is an official examinable part of the module. Study it as a connected explanation: define the central term, identify the related elements, explain their relationship, and finish with a relevant engineering use or limitation.

Learning objectives

- Define and scope MO4.1 — Exception handling (4 hours) using the official terminology retained above.
- Organise the important elements of MO4.1 — Exception handling (4 hours) into a logical sequence, classification, structure, or calculation.
- Connect MO4.1 — Exception handling (4 hours) with one engineering decision, operation, measurement, or safety requirement in the context of Files and Exception Handling.
- Write an examination answer on MO4.1 — Exception handling (4 hours) with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading MO4.1 — Exception handling (4 hours). It is a study organisation method, not an additional official syllabus requirement.

- Start with the exact technical term and its scope.
- Separate the topic into named elements, stages, types, or conditions.
- Explain the relationship using cause-and-effect, sequence, comparison, or a labelled representation.
- Apply the idea to a realistic engineering situation and state one limitation or common mistake.

Engineering application lens

The engineering value of MO4.1 — Exception handling (4 hours) is understood by asking what requirement it satisfies, what input or condition it depends on, how the output is obtained, and how a technician or designer verifies the result. Use the module context to make the application specific rather than memorising an isolated definition.

Worked answer method

Given: the topic, operating condition, diagram, data, or situation stated in the question.
Required: definition, explanation, comparison, procedure, calculation, or application as requested.
Method: organise the answer under principle, named elements, sequence or relationship, and application.
Answer: use the requested technical terms, units, labels, assumptions, and a concluding statement.

Exam answer blueprint

For a short-answer question on MO4.1 — Exception handling (4 hours), begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is MO4.1 — Exception handling (4 hours), and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining MO4.1 — Exception handling (4 hours)?
- How would you distinguish MO4.1 — Exception handling (4 hours) from a closely related idea?
- Where would an engineer or technician encounter MO4.1 — Exception handling (4 hours), and what must be checked before using it?
- What common mistake would make an answer or operation involving MO4.1 — Exception handling (4 hours) incorrect?

Common mistakes and safety emphasis

- Copying a definition without explaining the mechanism or purpose.
- Listing terms without showing their relationship.
- Using an example that does not match the stated topic or condition.
- Leaving out a diagram, unit, assumption, limitation, or safety point when the question requires it.

Malayalam support: MO4.1 — Exception handling (4 hours) □□□□ topic □□□□□□□□□□□□□ □□□□□□ exact technical term □□□□□□□□□□□□□□□. □□□□□ definition □□□□□□□□□□□ principle, named parts/steps, application, limitation □□□□□□□ □□□□□□□□□ □□□□□□□□□. English terminology, symbols, units, diagram labels □□□□□□□ □□□□□□□□□□. Exam answer □□□□□□□□□□□ concept-□□□□□ □□□□□□–□□□□ □□□□□□ □□□□□□□□□□□□□□□.

MO4.2 — Create, open and close a file (3 hours)

Concept and explanation

A file is persistent storage identified by a path. `open(path, mode, encoding="utf-8")` returns a file object; common modes are r for read, w for overwrite/create, x for exclusive creation, and a for append. The `with open(...) as f:` context manager closes the file automatically, even when an exception occurs.

Malayalam support: File open □□□□□□□□□□□□ mode □□□□ □□□□□□□□□□□: w □□□ content overwrite □□□□□□□□, a □□□□□□ □□□□□□□□□, r □□□□□□□□□□. `with` □□□□□□□□□□□□ file automatic □□□ close □□□□□□□□.

Study points

- Use UTF-8 for text; use a relative or validated path; never use w when data must be preserved; close resources deterministically.

Engineering / workplace application: Logs, student records, configuration files, and generated reports require file operations.

Illustrative example: `with open("notes.txt","w",encoding="utf-8") as f: f.write("Hello ")` creates or replaces the text file and closes it.

Common mistake: Opening with w accidentally erases the file; assuming a relative path refers to the same folder in every environment is unsafe.

Handbook treatment

MO4.2 — Create, open and close a file (3 hours) is an official examinable part of the module. Study it as a connected explanation: define the central term, identify the related elements, explain their relationship, and finish with a relevant engineering use or limitation.

Learning objectives

- Define and scope MO4.2 — Create, open and close a file (3 hours) using the official terminology retained above.
- Organise the important elements of MO4.2 — Create, open and close a file (3 hours) into a logical sequence, classification, structure, or calculation.
- Connect MO4.2 — Create, open and close a file (3 hours) with one engineering decision, operation, measurement, or safety requirement in the context of Files and Exception Handling.
- Write an examination answer on MO4.2 — Create, open and close a file (3 hours) with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading MO4.2 — Create, open and close a file (3 hours). It is a study organisation method, not an additional official syllabus requirement.

- Start with the exact technical term and its scope.
- Separate the topic into named elements, stages, types, or conditions.
- Explain the relationship using cause-and-effect, sequence, comparison, or a labelled representation.
- Apply the idea to a realistic engineering situation and state one limitation or common mistake.

Engineering application lens

The engineering value of MO4.2 — Create, open and close a file (3 hours) is understood by asking what requirement it satisfies, what input or condition it depends on, how the output is obtained, and how a technician or designer verifies the result. Use the module context to make the application specific rather than memorising an isolated definition.

Worked answer method

Given: the topic, operating condition, diagram, data, or situation stated in the question.
Required: definition, explanation, comparison, procedure, calculation, or application as requested.
Method: organise the answer under principle, named elements, sequence or relationship, and application.
Answer: use the requested technical terms, units, labels, assumptions, and a concluding statement.

Exam answer blueprint

For a short-answer question on MO4.2 — Create, open and close a file (3 hours), begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is MO4.2 — Create, open and close a file (3 hours), and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining MO4.2 — Create, open and close a file (3 hours)?
- How would you distinguish MO4.2 — Create, open and close a file (3 hours) from a closely related idea?
- Where would an engineer or technician encounter MO4.2 — Create, open and close a file (3 hours), and what must be checked before using it?
- What common mistake would make an answer or operation involving MO4.2 — Create, open and close a file (3 hours) incorrect?

Common mistakes and safety emphasis

- Copying a definition without explaining the mechanism or purpose.
- Listing terms without showing their relationship.
- Using an example that does not match the stated topic or condition.
- Leaving out a diagram, unit, assumption, limitation, or safety point when the question requires it.

Malayalam support: MO4.2 — Create, open and close a file (3 hours) □□□□ topic □□□□□□□□□□□□□ □□□□□ exact technical term □□□□□□□□□□□□□□. □□□□ definition □□□□□□□□□□ principle, named parts/steps, application, limitation □□□□□□ □□□□□□□□ □□□□□□□□. English terminology, symbols, units, diagram labels □□□□□□ □□□□□□□□□. Exam answer □□□□□□□□□□ concept-□□□□ □□□□□–□□□ □□□□□ □□□□□□□□□□□□□.

MO4.3 — Append a file and related functions (3 hours)

Concept and explanation

Appending preserves existing content and writes new data at the end. `read`, `readline`, `readlines`, iteration, `write`, and `writelines` are common operations. A robust program checks file existence, uses newline conventions consistently, and verifies the result. For structured data, a simple delimiter or a standard format should be chosen deliberately.

Malayalam support: Append mode □□□ content □□□□□□□□□□ □□□□□ record □□□□□□ □□□□□□□□. □□□□□□□□□□□ □□□□□□□□ newline, encoding, format □□□□□□ □□□□□□□□□□□□□.

Study points

- Append one complete record per line; ensure separators are present; read back after writing during testing; use a clear file format.

Engineering / workplace application: Daily transaction logs and attendance records are typical append-only files.

Illustrative example: `with open("log.txt","a",encoding="utf-8") as f: f.write("2026-08-15,OK ")` appends one record.

Common mistake: Calling read after the file pointer has reached the end returns an empty string unless seek(0) is used.

Handbook treatment

MO4.3 — Append a file and related functions (3 hours) is an official examinable part of the module. Study it as a connected explanation: define the central term, identify the related elements, explain their relationship, and finish with a relevant engineering use or limitation.

Learning objectives

- Define and scope MO4.3 — Append a file and related functions (3 hours) using the official terminology retained above.
- Organise the important elements of MO4.3 — Append a file and related functions (3 hours) into a logical sequence, classification, structure, or calculation.
- Connect MO4.3 — Append a file and related functions (3 hours) with one engineering decision, operation, measurement, or safety requirement in the context of Files and Exception Handling.
- Write an examination answer on MO4.3 — Append a file and related functions (3 hours) with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading MO4.3 — Append a file and related functions (3 hours). It is a study organisation method, not an additional official syllabus requirement.

- Start with the exact technical term and its scope.
- Separate the topic into named elements, stages, types, or conditions.
- Explain the relationship using cause-and-effect, sequence, comparison, or a labelled representation.
- Apply the idea to a realistic engineering situation and state one limitation or common mistake.

Engineering application lens

The engineering value of MO4.3 — Append a file and related functions (3 hours) is understood by asking what requirement it satisfies, what input or condition it depends on, how the output is obtained, and how a technician or designer verifies the result. Use the module context to make the application specific rather than memorising an isolated definition.

Worked answer method

Given: the topic, operating condition, diagram, data, or situation stated in the question.
Required: definition, explanation, comparison, procedure, calculation, or application as requested.
Method: organise the answer under principle, named elements, sequence or relationship, and application.
Answer: use the requested technical terms, units, labels, assumptions, and a concluding statement.

Exam answer blueprint

For a short-answer question on MO4.3 — Append a file and related functions (3 hours), begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is MO4.3 — Append a file and related functions (3 hours), and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining MO4.3 — Append a file and related functions (3 hours)?
- How would you distinguish MO4.3 — Append a file and related functions (3 hours) from a closely related idea?
- Where would an engineer or technician encounter MO4.3 — Append a file and related functions (3 hours), and what must be checked before using it?
- What common mistake would make an answer or operation involving MO4.3 — Append a file and related functions (3 hours) incorrect?

Common mistakes and safety emphasis

- Copying a definition without explaining the mechanism or purpose.
- Listing terms without showing their relationship.
- Using an example that does not match the stated topic or condition.
- Leaving out a diagram, unit, assumption, limitation, or safety point when the question requires it.

Malayalam support: MO4.3 — Append a file and related functions (3 hours) □□□□ topic □□□□□□□□□□□□ □□□□□ exact technical term □□□□□□□□□□□□□. □□□□ definition □□□□□□□□□□ principle, named parts/steps, application, limitation □□□□□□ □□□□□□□□ □□□□□□□□. English terminology, symbols, units, diagram labels □□□□□□ □□□□□□□□□. Exam answer □□□□□□□□□□ concept-□□□□ □□□□□–□□□ □□□□□ □□□□□□□□□□□□□.

Module summary: A reliable program validates input, manages resources, and preserves data through deliberate file modes.

Formula and Notation Bank

Remainder test

```
n % 2 == 0
```

Meaning: n is the integer; % returns the remainder. The result is Boolean and has no physical unit.

Average of a non-empty list

```
average = sum(values) / len(values)
```

Meaning: the numerator is the total and the denominator is the item count.

Diagram Library for Rapid Revision

e module to view its labelled learning-map diagram.

Module 2: Functions and

elled learning-map diagram.

Module 3: Lists, Dictionaries, Tuples and Sets

iew its labelled learning-map diagram.

Module 4: Files and

Open the module to view its labelled learning-map diagram.

Official Activities and Practical Requirements

Official activities / self-learning

- Write and run one small program after each topic.
- Maintain a practical record containing problem statement, algorithm, source code, test data, output, and short reflection.
- Test boundary cases such as zero, negative values, empty strings, and empty collections.

Practical requirements / equipment

- A Python 3 interpreter or IDLE/terminal environment; the syllabus does not prescribe a specific software package.

Assessment Methodology

Official assessment information is reproduced below from the supplied syllabus.

- The PDF lists a Lab Exam of 3 hours but does not provide a separate CIE/ESE mark distribution or question pattern. This lesson therefore treats the practice paper as non-official.
- Use the practical record, executable programs, output verification, and viva preparation as study evidence; the exact institutional marking scheme must be confirmed with the department.

Module-wise Questions and Answers

module-1 — Python Syntax, Operator Precedence and Control Flow

1. Explain Operator precedence. [3 marks]

Python evaluates an expression according to a precedence hierarchy. Parentheses are evaluated first, followed by exponentiation, unary operators, multiplication/division/floor division/modulus, addition/subtraction, comparisons, membership or identity tests, and Boolean not, and, or. When an expression is not immediately clear, parentheses are the safest way to communicate the intended order. Assignment is a statement rather than an arithmetic operator.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

2. Explain Decision control structures. **[3 marks]**

Decision structures choose a path based on a Boolean condition. Python uses if, elif, and else with indentation to delimit blocks. Conditions may combine comparisons with and, or, and not; nested decisions should be used only when they make the logic clearer. A complete program should consider equality, range boundaries, and invalid input.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

3. Explain Loop control structures. **[3 marks]**

A loop repeats a block while a pattern or condition remains valid. A for loop is suitable for iterating through a known sequence or range; a while loop is suitable when repetition depends on a changing condition. Initialise the control variable, update it, and make the exit condition reachable. break stops a loop, continue skips to the next iteration, and an else clause can run when a loop ends normally.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

module-2 — Functions and Strings

4. Explain Functions: swapping values and checking odd/even. **[3 marks]**

A function is defined with `def`, accepts parameters, performs a focused task, and may return a result. In Python, assignment can swap two variables as `a, b = b, a`, without a temporary variable. An odd/even function can use the remainder operator: n % 2 is zero for an even integer. The function should be tested with positive, negative, and zero inputs.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

5. Explain Default arguments. [3 marks]

A default argument supplies a value when the caller omits that parameter. It is written after required parameters, for example `def greet(name, message="Welcome")`. Default values should be immutable or deliberately handled; a mutable default list can retain data between calls and cause surprising behaviour.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

6. Explain Strings. [3 marks]

A string is an ordered, immutable sequence of characters. Indexing starts at zero, negative indices count from the end, and slicing uses start:stop:step with an exclusive stop. Common methods include lower, upper, strip, split, join, replace, find, and count. Concatenation and f-strings format text; input arrives as a string and may need conversion.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

module-3 — Lists, Dictionaries, Tuples and Sets

7. Explain Lists. [3 marks]

A list is an ordered, mutable collection written with square brackets. It may contain mixed data types, supports indexing and slicing, and provides append, extend, insert, remove, pop, sort, and reverse operations. Iterating over a list is clearer than repeatedly writing separate variables. Copying with `b=a` aliases the same list; `a.copy()` creates a shallow copy.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

8. Explain Dictionaries. [3 marks]

A dictionary stores key-value pairs. Keys must be hashable and unique; values may be any suitable object. Access can use `d[key]` when the key must exist or `d.get(key, default)` when absence is expected. Items, keys, and values views support iteration. Dictionaries are useful for records and fast lookup.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

9. Explain Lists: nested and practical operations. **[3 marks]**

The supplied outline repeats the word lists for MO3.3. This lesson preserves that official label and uses the topic for nested lists, list comprehensions, filtering, and practical traversal. A nested list is a list whose elements are themselves lists; access requires one index per level.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

10. Explain Sets. **[3 marks]**

A set is an unordered collection of unique hashable elements. It supports add, remove/discard, union, intersection, difference, and membership testing. Sets are ideal for eliminating duplicates, but they do not preserve a meaningful display order.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

module-4 — Files and Exception Handling

11. Explain Exception handling. **[3 marks]**

An exception is an event that interrupts normal execution, such as ValueError, TypeError, IndexError, KeyError, or FileNotFoundError. A try block contains risky operations; except handles a known failure; else runs when no exception occurs; finally runs for cleanup. Catch specific exceptions rather than using a bare except, and do not use exception handling to mask incorrect logic.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

12. Explain Create, open and close a file. [3 marks]

A file is persistent storage identified by a path. `open(path, mode, encoding="utf-8")` returns a file object; common modes are r for read, w for overwrite/create, x for exclusive creation, and a for append. The `with open(...) as f:` context manager closes the file automatically, even when an exception occurs.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

13. Explain Append a file and related functions. [3 marks]

Appending preserves existing content and writes new data at the end. `read`, `readline`, `readlines`, iteration, `write`, and `writelines` are common operations. A robust program checks file existence, uses newline conventions consistently, and verifies the result. For structured data, a simple delimiter or a standard format should be chosen deliberately.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

Practice Model Question Paper

Practice Model Paper — Not an Official Examination Paper

This paper is generated from the supplied module coverage for revision and does not claim to reproduce the official examination pattern.

1. **1.** Define or explain *Operator precedence* with one relevant application. (5 marks)
2. **2.** Define or explain *Decision control structures* with one relevant application. (5 marks)
3. **3.** Define or explain *Functions: swapping values and checking odd/even* with one relevant application. (5 marks)
4. **4.** Define or explain *Default arguments* with one relevant application. (5 marks)
5. **5.** Define or explain *Lists* with one relevant application. (5 marks)
6. **6.** Define or explain *Dictionaries* with one relevant application. (5 marks)
7. **7.** Define or explain *Exception handling* with one relevant application. (5 marks)
8. **8.** Define or explain *Create, open and close a file* with one relevant application. (5 marks)

> **Writing advice:** Use headings, standard terminology, and labelled diagrams or procedures where relevant.

Quick Revision

Module-wise key points

- **Python Syntax, Operator Precedence and Control Flow:** Syntax supports expressions; decisions select one path; loops express controlled repetition.
- **Functions and Strings:** Functions make solutions reusable; strings make text processing predictable and testable.
- **Lists, Dictionaries, Tuples and Sets:** Choose collections by required order, mutability, uniqueness, and access method.
- **Files and Exception Handling:** A reliable program validates input, manages resources, and preserves data through deliberate file modes.

Exam checklist

- Write the official terminology before adding explanation.
- Use a labelled diagram or step sequence when it improves the answer.
- Check conditions, boundaries, safety, hygiene, and documentation points.
- Separate official syllabus information from your own example.

> **Last-minute revision:** Review the coverage table, formula bank, diagram library, and common-mistake notes inside every topic.

Glossary

Technical glossary

Term	Short meaning

Term	Short meaning
Operator precedence	Python evaluates an expression according to a precedence hierarchy. Parentheses are evaluated first, followed by exponentiation, unary operators, multiplication/division/floor division/modulus, addition/subtraction, comparisons, membership or identity tests, a
Decision control structures	Decision structures choose a path based on a Boolean condition. Python uses if, elif, and else with indentation to delimit blocks. Conditions may combine comparisons with and, or, and not; nested decisions should be used only when they make the logic clearer.
Loop control structures	A loop repeats a block while a pattern or condition remains valid. A for loop is suitable for iterating through a known sequence or range; a while loop is suitable when repetition depends on a changing condition. Initialise the control variable, update it, and
Functions: swapping values and checking odd/even	A function is defined with `def`, accepts parameters, performs a focused task, and may return a result. In Python, assignment can swap two variables as `a, b = b, a`, without a temporary variable. An odd/even function can use the remainder operator: n % 2 is z
Default arguments	A default argument supplies a value when the caller omits that parameter. It is written after required parameters, for example `def greet(name, message="Welcome")`. Default values should be immutable or deliberately handled; a mutable default list can retain d
Strings	A string is an ordered, immutable sequence of characters. Indexing starts at zero, negative indices count from the end, and slicing uses start:stop:step with an exclusive stop. Common methods include lower, upper, strip, split, join, replace, find, and count.
Lists	A list is an ordered, mutable collection written with square brackets. It may contain mixed data types, supports indexing and slicing, and provides append, extend, insert, remove, pop, sort, and reverse operations. Iterating over a list is clearer than repeate
Dictionaries	A dictionary stores key-value pairs. Keys must be hashable and unique; values may be any suitable object. Access can use `d[key]` when the key must exist or `d.get(key, default)` when absence is expected. Items, keys, and values views support iteration. Dictio
Lists: nested and practical operations	The supplied outline repeats the word lists for MO3.3. This lesson preserves that official label and uses the topic for nested lists, list comprehensions, filtering, and practical traversal. A nested list is a list whose elements are themselves lists; access r
Sets	A set is an unordered collection of unique hashable elements. It supports add, remove/discard, union, intersection, difference, and membership testing. Sets are ideal for eliminating duplicates, but they do not preserve a meaningful display order.
Exception handling	An exception is an event that interrupts normal execution, such as ValueError, TypeError, IndexError, KeyError, or FileNotFoundError. A try block contains risky operations; except handles a known failure; else runs when no exception occurs; finally runs for cl
Create, open and close a file	A file is persistent storage identified by a path. `open(path, mode, encoding="utf-8")` returns a file object; common modes are r for read, w for overwrite/create, x for exclusive creation, and a for append. The `with open(...) as f:` context manager closes th
Append a file and related functions	Appending preserves existing content and writes new data at the end. `read`, `readline`, `readlines`, iteration, `write`, and `writelines` are common operations. A

Term	Short meaning
	robust program checks file existence, uses newline conventions consistently, and verifies the re

Official References and Resources

References listed in the supplied syllabus

1. Gowrishankar S and Veena A, Introduction to Python Programming, 1st Edition, CRC Press/Taylor & Francis, 2018.
2. Python3 for Absolute Beginners, Tim Hall and J. P. Stacey.
3. Head First Python: A Brain-Friendly Guide, Paul Barry.

Official learning resources listed

- https://www.w3schools.com/python/
- https://www.tutorialspoint.com/python/
- https://www.geeksforgeeks.org/python-programming-language/

In-page Search

Generated as a student lesson from the supplied syllabus PDF. Revision: Revision 2021 · Course code: 1257. Practice questions and explanations are educational support, not official examination material.